



הפקולטה להנדסה
המעבדה לעיבוד אותות

הנדסת chatbot והתאמתו לתחומי עניין ספציפיים במעבדה לעיבוד אותות

מרדכי לשנסקי
רון דרויאן

פרויקט שנה ד' לקראת תואר ראשון בהנדסה

מנחה: מר פיני טנדייטניק, מר יואב אלינסון
מנחה אקדמי: פרופ' שרון גנות

אוקטובר 2025

תודות

ברצוננו להביע את תודתנו והערכתנו לכל מי שליווה ותמך בנו לאורך הדרך בתהליך ביצוע פרויקט הגמר.

ראשית, תודה מיוחדת למנחים שלנו, מר פיני טנדייטניק ומר יואב אליסון, על ההכוונה, התמיכה, והידע הרב שהעניקו לנו. העצות והמשוב שלהם היוו בסיס משמעותי להצלחת הפרויקט.

תודה לפקולטה להנדסה ע"ש אלכסנדר קופקין ולפרופ' שרון גנות על מתן המשאבים, הציוד, והידע שאפשרו לנו לבצע את המחקר והפיתוח ברמה הגבוהה ביותר.

ברצוננו להודות גם לחברינו לכיתה ולעמיתים שסייעו לנו במהלך הדרך, אם בשיחות, עצות, או שיתוף פעולה.

תודה מיוחדת למשפחותינו ולקרובינו על התמיכה, הסבלנות והעידוד המתמיד לאורך כל שלבי הלימודים והפרויקט.

לבסוף, אנו רוצים להביע את הערכתנו לכל מי שתרום בדרכו להשלמת פרויקט זה – בין אם במעשה, בעצה טובה, או בהשראה.

תוכן עניינים

1	מבוא	5
1.1	תקציר	5
1.2	מטרת הפרויקט	5
1.3	דרישות	6
2	סקר ספרות	7
2.1	מודל שפה – LLM	7
2.1.1	מה זה LLM	7
2.1.2	איך עובד מודל שפה	7
2.1.3	פרמטרים של מודל שפה	8
2.1.4	פרמטרים של הפעלת מודל שפה	9
2.1.5	טוקנים מיוחדים	10
2.1.6	ממודל שפה לצא'טבוט	11
2.2	השוואה בין צ'אטבוטים	11
2.2.1	איך משווים בין מודלי שפה	11
2.2.2	בנצ'מארקים נפוצים	13
2.3	שיפור ביצועים של LLM	14
2.3.1	הנדסת פרומפטים	14
2.3.2	RAG	15
3	בדיקת המודלים	16
3.1	המבחנים שבצענו	16
3.2	המודלים שבדקנו	18
3.3	תוכנת הבדיקה	18
3.3.1	מבנה המערכת	19
3.3.2	זרימת העבודה	21
3.3.3	סיכום	21
3.4	תוצאות	22
3.5	בחירת המודל	27
4	תוכנת השיחה	28
4.1	דמויות	28
4.2	RAG	28
4.3	תוכנת CHATBOT עם RAG	29
4.4	מערכת ROS	30
4.5	דוגמאות הרצה	31
5	מסקנות מהפרויקט והצעות להמשך	33

33	השוואה בין מודלי שפה.....	5.1
33	שיחה עם CHATBOT מותאם אישית.....	5.2
35	נספחים.....	6
35	OLLAMA.....	6.1
35	קובץ מודל.....	6.1.1
36	LANGCHAIN.....	6.2
36	ROS.....	6.3
37	FAISS.....	6.4
37	מדריכים.....	6.5
37	התקנת והרצת בדיקות.....	6.5.1
38	הוספת מבחן לבדיקות.....	6.5.2
41	התקנת ROS בייתית.....	6.5.3
41	בניית ROS package.....	6.5.4
41	בניית מאגר RAG.....	6.5.5
42	התקנת והרצת הגרסה הביתית של מערכת הצ'אט.....	6.5.6
42	הרצת מערכת הצ'אט על הרובוט במעבדה.....	6.5.7
43	תיאור ה-GIT של הפרויקט.....	6.6
46	ביבליוגרפיה.....	7

1 מבוא

1.1 תקציר

בפרויקט זה עסקנו בהערכה של מודלי שפה (LLM) לפי מס' שיטות השוואה שונות ובשיטות לשיפורם, במטרה להטמיע אותם בצורה הטובה והיעילה ביותר בתוך מערכות שיאפשרו ניהול שיחה עם משתמשים.

במסגרת הפרויקט תחילה בנינו מערכת צ'אט מבוססת טקסט המיועדת למחשב האישי כאשר בחלקו השני של הפרויקט התאמנו והטמענו את המערכת ברובוט דמוי אדם מדגם ARI של חברת PAL Robotics תוך כדי הסבה של המערכת הקיימת לצ'אט קולי באופן שמדמה שיחה עם עוזר אישי אנושי.

הפרויקט מורכב משתי תוכנות מרכזיות:

1. **תוכנת בדיקה** - מבצעת הערכה למודלי שפה על-ידי כך שהיא מעבירה אותם מבחני השוואה מסוגים שונים, כדי לעזור למתכנתים למצוא את מודל השפה שהכי מתאים לדרישות שלהם.

2. **תוכנת שיחה** - מאפשרת למשתמש לקיים שיחה עם רובוט מומחה בתחום מוגדר, תוך שימוש ביכולות Speech-to-Text ו-Text-to-Speech, בהמשך תבחן הטמעה של אלגוריתם לעיבוד אות קול בזמן אמת שפותח במעבדה, וכמו כן ניהול זיכרון שיחה.

1.2 מטרת הפרויקט

ישנן שתי מטרות מרכזיות לפרויקט:

1. לספק מערכת שמאפשרת לטפל בבעיית ההערכה של מודלי שפה, כלומר מערכת שתבחר את המודל שיביא את השקלול המשתלם ביותר מבחינת ביצועים כאשר השאיפה המרכזית היא לאיכות תשובה הגבוהה ביותר - בהתבסס על בדיקות באמצעות מבחני השוואה קיימים, ולזמן תגובה קצר ככל הניתן. מטרה זו הינה חשובה במיוחד לאור המציאות הדינמית כיום בה מפותחים מודלים חדשים ומבחנים חדשים לבדיקה והשוואה של המודלים על בסיס שבועי. לאור זאת על-מנת לעמוד בקצב ההתקדמות הטכנולוגית המערכת תאפשר להוסיף מודלים חדשים ומבחנים חדשים בקלות לרשימת הבדיקות.

2. לבנות מערכת שמאפשרת להפוך את מודל השפה הנבחר למומחה בתחום מסוים, ולאפשר לנהל איתו שיחה תוך שימוש בכלים של text to speech ו speech to text, כאשר גם כאן חלק מהמטרה היא לאפשר הוספה/שינוי של תחומי מומחיות בקלות, ולשנות את בחירת המודל בו משתמשים עקב ההתקדמות הטכנולוגית. את מערכת זאת נרצה לשלב ברובוט של PAL Robotics, ARI.



איור [p1] תמונה של ARI.

1.3 דרישות

עבור תוכנת הבדיקה

1. גמישות:
התוכנה צריכה לאפשר בדיקה של מספר מרובה של מודלים במספר מרובה של מבחנים, עם אפשרות לשנות את הפרמטרים הרלוונטיים של הבדיקה בקלות.
2. קריאות:
התוצאות של הבדיקות צריכות להיות קלות לקריאה וניתנות להשוואה, כדי לאפשר למשתמש לבחור את המודל שמתאים לו בקלות.
3. מהירות וזיכרון:
הרצת בדיקות על מודל שפה הינו תהליך שיכול להיות יקר מאוד בזמן ומשאבים (זיכרון גרפי, עלויות חשמל), ולכן התוכנה צריכה לחסוך בכך איפה שניתן.

עבור תוכנת השיחה

1. מומחיות:
על התוכנה להפוך את מודל השפה למומחה בתחום מסוים עבורו תנוהל השיחה עם המשתמש, ובכך תגרום לו לספק מידע יותר רלוונטי ואמין למשתמש.
2. מהירות:
התוכנה צריכה לעבוד כמה שיותר קרוב לזמן אמת בכל השלבים שלה כדי לאפשר שיחה רציפה עם המשתמש.
3. היסטוריה:
כדי שהשיחה עם מודל השפה תרגיש כמו שיחה אמיתית על התוכנה לזכור את שאלות העבר של המשתמש כדי שתוכל להתייחס אליהם.

2 סקר ספרות

2.1 מודל שפה – LLM

2.1.1 מה זה LLM

LLM או Large Language model (מודל שפה גדול) הינה מערכת בינה מלאכותית שמטרתה להבין וליצור טקסט בשפות אנושיות, אשר נבנית בעיקר בעזרת שימוש בטכניקות של למידה עמוקה, רובם מבוססים בעיקר על ארכיטקטורת הטרנספורמר (Transformer). המשמעות היא שבאמצעות המרה של טקסט לייצוגים מספריים (ייצוג שכזה נקרא קידוד - Embedding), המודל לומד קשרים סטטיסטיים ומשמעויות סמנטיות בין מילים. תכונה זו מאפשרת למודלים לחזות וליצור טקסט חדש בצורה שנראית אנושית.

[1]

2.1.2 איך עובד מודל שפה

כדי להבין כיצד LLM עובד באופן מופשט נסקור את תהליך האימון:

בצורה כללית מאוד מודלי שפה מאומנים על ידי מעבר על כמויות עצומות של מידע לאימון (טקסט) כאשר תהליך הלמידה מורכב מ-2 שלבים:

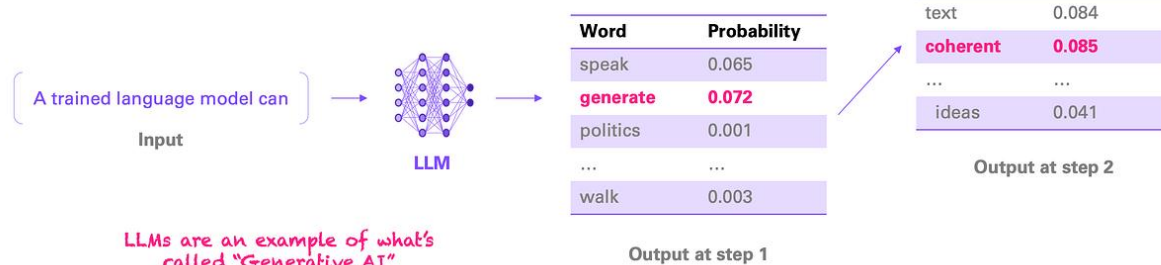
1. **אימון ראשוני (Pre-training):** הטקסט מקודד ומוזן לתוך רשת הטרנספורמר (חלק זה מכונה כה"context") כאשר ביציאה מהרשת המודל מפיק התפלגות הסתברותית עבור כל מילה אפשרית במילון כדי לחזות את הטוקן הבא ברצף. כדי למדוד את הצלחת החיזוי, המודל משווה את התחזית שלו למילה האמיתית באמצעות פונקציית הפסד (Loss Function), ומעדכן את הפרמטרים שלו כדי למזער את הטעות בעזרת תהליך שנקרא Gradient Descent. לאחר מכן מתווספת המילה האמיתית הבאה לרצף (הקונטקסט גדל) והמודל צריך לחזות את המילה הבאה. באופן זה המודל לומד בהדרגה את מבנה ותחביר השפה ובנוסף "צובר" כמות גדולה של ידע על העולם.

2. **התאמה ייעודית / כוונן עדין (Fine-tuning):** בשלב זה המודל מאומן על סט נתונים קטן וממוקד יותר, שנבחר בקפידה, כדי לשפר את ביצועיו במשימות מסוימות או תחומים ייחודיים. שלב זה מתבצע באופן דומה לשלב הקודם, רק שמידע האימון נבחר בצורה ידנית על מנת לשפר את הביצועים של המודל במשימות ספציפיות.

מסקירת תהליך זה נוכחנו לדעת שלמרות המורכבות של מודלי השפה בסופו של דבר הם מנגנונים שמבצעים חיזוי של המילה הבאה בהתבסס על רצף המילים הקודם, וחשוב להבין שהם אינם "מבינים" את השפה כמו בני אדם, וגם אינם מתייחסים למילים ברמה של אותיות, אלא ברמה של טוקנים (Tokens) – יחידות טקסט מקודדות ואינם "רואים" את המילים כאותיות אלא כמספרים הניתנים לעיבוד סטטיסטי. בפועל, כל פעולתם מסתכמת בחיזוי הטוקן הבא בהתבסס על הרצף הקודם.

Natural language generation

After training: We can generate text by predicting one word at a time



איור [p2]: תמונה שממחישה את הפלט של מודל שפה.

2.1.3 פרמטרים של מודל שפה

למודלי שפה יש המון פרמטרים נסביר בקצרה על החשובים, ושדרך כלל חשופים למשתמש.

מספר פרמטרים (Weights) – מספר המשקולות שיש למודל. כמות גדולה יותר של פרמטרים בדרך כלל מעידה על יכולת טובה יותר לתפוס דפוסים מורכבים בתוך שפה, אך גם זה אומר שהדרישות החישוביות של האימון וההרצה של המודל יהיו גדולות יותר. חשוב לציין שכמות גדולה יותר של פרמטרים לא מבטיחה ביצועים יותר של המודל אך היא מגדילה את ה"פוטנציאל" של המודל לביצועים טובים יותר.

חלון הקונטקסט / אורך הקונטקסט (Context Window) – מגדיר את מספר הטוקנים אשר המודל מסוגל "לראות" בכל רגע נתון. כפי שראינו התפקיד של המודל הוא לחזות את הטוקן הבא בהינתן קונטקסט אז גודל הקונטקסט קובע את הגודל המקסימלי למשימת חיזוי, כאשר חלון גדול יותר אומר שהמודל יוכל להתייחס ליותר מידע בכל חיזוי. גם כאן חלון גדול יותר מגדיל את הדרישות החישוביות של המודל. בנוסף חשוב לציין שבמודלים שעובדים בארכיטקטורת decoder-only (רוב המודלים המוכרים) גודל הקונטקסט כולל בתוכו גם את אורך הקלט וגם את אורך הפלט ובצורה.

אורך הקידוד (Embedding Size) – קובע את גודל הווקטור שכל תוקן מתורגם אליו, אורך אמבדינג גדול יותר מאפשר ייצוג עשיר יותר לכל טוקן ולכן מאפשר למודל "הבנה" יותר נכונה של כל אחד. גם הוא מגדיל את הדרישות החישוביות של המודל.

2.1.4 פרמטרים של הפעלת מודל שפה



איור [p3] הפרמטרים של מודל שפה.

Temperature - פרמטר שמוסרת את האקראיות או החד-פעמיות של הטקסט שהמודל מפק. טמפרטורה נמוכה (למשל קרוב ל-0) תגרום לתשובות צפופות, עקביות ודטרמיניסטיות; טמפרטורה גבוהה תאפשר יצירתיות רבה יותר, אפשרות לבחור טוקנים פחות סבירים, ולכן תוצאה פחות צפוייה אך מגוונת יותר.

Max Tokens - מגביל את מספר הטוקנים שהמודל יפיק כהמשך לתשובה. מאפשר לשלוט באורך הפלט, למנוע תשובות שמתרבות יתר על המידה או יציבות זמן הריצה. חשוב לציין שהמודל לא יפיק תשובה קצרה יותר אלא התשובה פשוט תחתך כשהיא תגיע למספר הטוקנים המקסימלי.

Top-p (Nucleus Sampling) - פרמטר שמגדיר סף הסתברות מצטברת שבו מתבצעת הבחירה של הטוקנים הבאים. לדוגמה, אם $P = 0.9$ המודל יבחר את הטוקנים מתוך קבוצת הטוקנים שיש להם הסתברות כוללת של 90%. זה מאפשר איזון בין יצירתיות ורלוונטיות, מכיוון ששאר הטוקנים עם הסתברויות נמוכות מאוד לא ייחשבו כלל.

Top-k - פרמטר שקובע שהמודל יבחר רק מתוך K הטוקנים עם ההסתברויות הגבוהות ביותר בכל צעד, במקום לבחור מתוך כל הטוקנים האפשריים. ככל שה K קטן הפלט יהיה צפוי וממוקד יותר, וככל שהוא גדול אפשרות הגיון תהיה גדולה יותר.

Presence Penalty - עונש שמטרתו למנוע חזרתיות בטקסט: כלומר הימנעות מחזרה של מילים או רעיונות שכבר הופיעו. פרמטר זה "מעודד" את המודל לגוון יותר וליצור טקסט פחות חוזר על עצמו.

Frequency Penalty - מקטין את ההסתברות שהמודל ישתמש באותם מילים נפוצות שוב ושוב. עונש זה מעודד את המודל לא להשתמש באותם ביטויים או מילות קישור שוב ושוב.

[2]

2.1.5 טוקנים מיוחדים

טוקנים מיוחדים (Special Tokens) הם סמלים ייחודיים שנוספו לאוצר המילים של מודל השפה כדי לסמן גבולות, תפקידים או מצבים מיוחדים בתהליך העיבוד. שלא כמו טוקנים רגילים (שמייצגים מילים או חלקי מילים), הם משמשים כמעין "אותות בקרה" שמכוונים את המודל כיצד לפרש את הקלט או כיצד להפיק את הפלט.

להלן כמה דוגמאות לשימושים של טוקנים מיוחדים:

role prompt – בלוק שלו המודל ייחס חשיבות מסוימת בדרך כלל יש שלושה תפקידים:

- **system** – ההוראות כלליות למודל, שניתנות בדרך כלל על ידי המפתחים. המודל אומן בשלב Fine-tuning לייחס להן חשיבות גבוהה במיוחד.
- **user** – מייצג את השאלה או הפקודה שהמשתמש שולח.
- **assistant** – האזור שבו המודל מספק את התשובה, או תשובות שנתן בעבר. המודל בדרך כלל נחשף לטוקנים אלו בשלב Fine-tuning.

EOS – טוקנים שמסמנים את תחילת וסוף התשובה. בלעדיהם, האלגוריתם היה ממשיך לבקש מהמודל לייצר עוד ועוד טקסט בלי סוף. לכן כבר בשלב ה-Pre-training, מאמנים את המודל להשתמש בטוקן זה כדי לדעת היכן לעצור.

חשוב לציין שהטוקנים המיוחדים לא בהכרח זהים בין מודלים שונים, דבר שגורם לכך שפרומט מסויים יכול להפיק תוצאה מצוינת עבור מודל אחד ותוצאה לא טובה עבור מודל אחר למרות שהמודל שני אולי יותר טוב מהראשון.

דוגמא לשימוש בטוקנים מיוחדים במודל Llama 3:

• Single User Message:

```
<|begin_of_text|><|start_header_id|>user<|end_header_id|>

What is France's capital?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

• System Message and Multi-Turn Conversation:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful AI assistant for travel tips and
recommendations.<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is France's capital?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>

The capital of France is Paris.<|eot_id|>
```

איור [p4] דוגמא לטוקנים מיוחדים.

2.1.6 ממודל שפה לצא'טבוט

כדי להפוך את מודל השפה למערכת שמנהלת שיחה (צ'אטבוט) – כלומר, שיחה רציפה, טבעית ומשמעותית עם משתמש – יש צורך להוסיף "מעטפת תוכנה" שמטפלת באינטראקציה: בקלט של המשתמש, ביצירת תגובות, ובניהול של הקשר השיחה (היסטוריית השיחה).

העטיפה הזו כוללת רכיבים מרכזיים כגון:

ניהול זיכרון: היכולת לזכור את השאלות והתגובות הקודמות, לא רק את השאלה הנוכחית. כך הצ'אטבוט שומר על קוהרנטיות – יודע להתייחס לדברים שנאמרו קודם.

הרחבה של ההקשר (context window): מה נכנס כקלט למודל בכל סבב – כוללים את כל ההיסטוריה או רק חלק ממנה.

ניהול זיכרון לטווח ארוך: טכניקות שמאפשרות לשמור מידע חשוב לאורך זמן (לא רק בתגובה המיידית), כמו סיכום של שיחות, אחסון במסדי נתונים ייעודיים, או אחזור קונטקסט רלוונטי לפי הצורך.

הגבלת עקביות ויעילות: ככל שהשיחה מתארכת, גדל מספר התווים (tokens) שיש להעביר למודל, מה שמעלה את זמן התגובה ואת עלות החישוב. לכן יש צורך באסטרטגיות שמסירות או מצמצמות את חלקי השיחה הפחות חשובים, למשל: שמירת רק ההודעות האחרונות, סיכומים קצרים, או זיהוי קטעי שיחה רלוונטיים.

יוצא מכך ש, "צ'אטבוט מבוסס LLM" הוא למעשה שילוב של:

ה-LLM, מנגנוני שימור והזכרה של היסטוריית השיחה והקשר השימוש ולוגיקה שמבצעת את הממשק עם המשתמש (לקיחת קלט, עיבוד, והחזרת פלט) באופן שמאפשר שיחה טבעית ואנושית יותר.

[3]

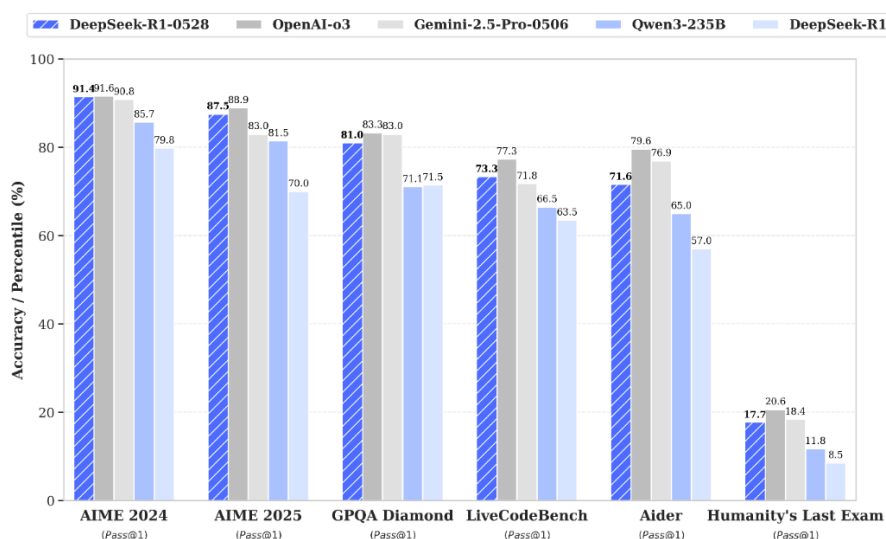
2.2 השוואה בין צ'אטבוטים

2.2.1 איך משווים בין מודלי שפה

- הערכה מבוססת משתמשים אמיתיים (Human Evaluation) – במקום להתבסס על מדדים יבשים, בוחנים את המודלים באמצעות הערכה אנושית לדוגמא:
 1. דירוג איכות התשובה מ-1 עד 5.
 2. מבחן A/B - מציגים למשתמש שתי תשובות ממודלים שונים ומבקשים לבחור איזו טובה יותר.
 3. מבחן "בליינד" – המשתמש לא יודע איזה מודל הפיק כל תשובה, למניעת הטיה.
- הדרך הנפוצה ביותר להשוואת ביצועי מודלים של צ'אטבוטים היא באמצעות הרצה על בנצ'מארקים ייעודיים – מבחנים המבוססים על מערכי נתונים סטנדרטיים שנועדו למדוד ביצועים במשימות שונות.

סוג זה של הערכה מתמקד בחוויה האמיתית של המשתמש ומאפשר לזהות בעיות עדינות כמו סגנון, טון, או היכולת לשמור על הקשר (Context). לעומת זאת, חסרונותיה הם שתהליך זה דורש אנשים שיבדקו ידנית ולכן יקר בזמן ובכסף, וכמו כן התוצאות עלולות להיות סובייקטיביות.

- בודקים את המודל כחלק ממערכת שלמה ולא רק כתשובות בודדות. בוחנים כיצד הוא מתפקד בסביבת עבודה אמיתית, למשל בתוך צ'אטבוט, אפליקציה רפואית או מערכת חיפוש. מדדים אפשריים הם זמן תגובה (Latency), צריכת זיכרון ומשאבים, אחוזי הצלחה במשימות מוגדרות מראש כגון מספר הפניות שנפתרו ללא צורך במענה אנושי, ושביעות רצון כללית של משתמשי קצה. גם צורת השוואה זו דורשת זמן ומשאבים רבים. חיסרון נוסף הוא שקשה לבודד את השפעת המודל מהשפעת רכיבים אחרים במערכת. יתרונה הוא שהיא משקפת באופן הטוב ביותר את השימוש האמיתי.
- השוואת עלויות ויעילות: זמן ריצה לתשובה אחת, כמות זיכרון נדרשת, צריכת GPU/CPU, עלות כספית להפעלת המודל לאורך זמן. מדדים לדוגמה: Tokens per second – כמות הטוקנים שהמודל מפיק בשנייה, עלות לכל 1,000 טוקנים, אנרגיה לצורך הפעלה. שיטת השוואה זו מתאימה לארגונים שמעוניינים במודל קל ויעיל, ומאפשר בחירה מושכלת בהתאם לתקציב. חסרונותיה של שיטה זו הם שלא מודדים את איכות התשובות ישירות, ושמודל זול ויעיל עלול לתת תשובות פחות מדויקות.
- הערכה איכותית – בדיקה ידנית ומעמיקה של סוגי שגיאות ספציפיים: האם המודל ממציא עובדות (Hallucinations), האם המודל שומר על הקשר (Context) בשיחה ארוכה, האם המודל עוקב אחרי פעולות מורכבות. שיטת השוואה זו מאפשרת הבנה עמוקה של החוזקות והחולשות אך היא דורשת בדיקה ידנית כך שהיא תלויה במומחיות של הבודק, סובייקטיבית ואינה ניתנת להפעלה על כמויות גדולות של נתונים.



איור [p5]: גרף השוואת מודלי שפה.

מהו בנצ'מארק למודל שפה?

בנצ'מארק (בוחן ביצועים) הוא מאגר של משימות או שאלות שמאפשרים להעריך את איכות המודל בצורה עקבית ואובייקטיבית. כל בנצ'מארק מספק שיטה אחידה למדידת ביצועי המודל, כך שניתן להשוות בין מודלים שונים באופן הוגן.

כיצד מודדים ציון בבנצ'מארק מסוים

זה תלוי בבנצ'מארק אך יש כמה שיטות נפוצות

1. הבנצ'מארק מורכב משאלות אמריקאיות – רק צריך לחלץ את התשובה הסופית

2. הבנצ'מארק מורכב משאלות עם תשובה אחת נכונה – אפשר לחשב "exact match" האם המודל החזיר בדיוק את התשובה הנכונה ו"F1 score" כמה מתוך התשובה של המודל היא התשובה הנכונה. יש גם שיטות נוספות כמו לבדוק כמה האמבדינג של התשובה של המודל קרוב לאמבדינג של התשובה הנכונה
3. הבנצ'מארק יכול להיות מורכב משאלות עם יותר מתשובה אחת נכונה או אפילו שהתשובה הנכונה לא בהכרח מוגדרת - אפשר לתת לאדם לעבור על התשובות ולדרג אותם, אופציה נוספת פופלארית היא "LLM as judge" בה מבקשים ממודל גדול וחזק לדרג את התשובות לכל שיטה יש יתרונות וחסרונות הנה כמה מהם:

שיטה	יתרונות	חסרונות
1. אמריקאי	קל ומהיר לבדיקה בצורה מדויקת	דרישות המבחן לא מאפשרות באמת לבדוק כיצד המודל יבצע באינטרקציות עם אנשים. מגבילים את המודל לענות בפורמט מאוד מסוים וזה יכול לפגוע בביצועים שלו.
2. תשובה אחת	בדיקה יחסית קלה	דרישה של שאלה עם תשובה ספציפית גם מאוד מגבילה את סוג השאלות שאפשר לשאול, ככל שהתשובה יותר ארוכה, הבדיקה פחות מדויקת
3. תשובה פתוחה	מאפשר מבחן רחב של מבחנים שיעדיו על תכונות שונות של מודל השפה מאפשר בדיקה שדומה מאוד לשימוש במודל	בדיקה קשה שדורשת מודל שפה חזק או התערבות אנושית

2.2.2 בנצ'מארקים נפוצים

The Stanford Question Answering Dataset - **SQuAD**

בנצ'מארק הבודק את יכולות הבנת הנקרא של המודל. הבנצ'מארק מתבסס על אוסף עצום של 107,785 שאלות ותשובות המבוססות על 536 מאמרים מוויקיפדיה. ברוב המקרים ההערכה מתבצעת בשיטת Exact Match ו-F1 Score (שיטה מס' 2).

Multi-step Soft Reasoning - **MuSR**

בנצ'מארק המבוסס על אוסף של 750 שאלות אמריקאיות שמטרתו להעריך את יכולות ההגיון והחשיבה – פתרון בעיות מורכבות(chain of thought) של מודלי LLM. השאלות מורכבות מבלוק ארוך של טקסט, ומתחלקות לשלושה תחומים: תעלומות רצח, מיקום אובייקטים, וחלוקת עבודה.

Massive Multitask Language Understanding - **MMLU-PRO / MLLU**

בנצ'מארק המודד את רוחב הידע הכללי של המודל ומבוסס על מקבץ של שאלות אמריקאיות בעלות 4 אופציות ו-10 אופציות בגרסת ה-PRO במגוון רחב של תחומים שמטרתו לבדוק את הידע שהמודל "מכיל" וכמה הוא עונה על שאלות נכונה במגוון רחב של תחומים. הגרסה הרגילה מכילה 16,000 שאלות ב-57 תחומים ואילו ה-PRO מכילה 12,000 שאלות ב-14 תחומים.

Measuring How Models Mimic Human Falsehoods - TruthfulQA

בנצ'מארק ייחודי הבודק עד כמה המודל מושפע ממידע שגוי או מוטה. מבוסס על אוסף של 817 שאלות ב-38 תחומים כאשר השאלות הן שאלות שבני אדם עלולים לטעות בהם בגלל אמונה או תפיסה שגויה. מטרתו לבדוק כמה המודל מצליח לענות נכונה על שאלות שיכול להיות שבתהליך האימון שלו הוא קיבל מידע שגוי לגביהם, יש כמה דרכים להריץ את המבחן: שאלות אמריקאיות עם תשובה אחת נכונה, שאלות אמריקאיות עם מספר תשובות נכונות, או סתם שאלות פתוחות במקרה זה אפשר להשתמש בשילוב של שיטה מס' 2 ושיטה מס' 3 כדי לבדוק אם תשובה נכונה.

multi-turn benchmark - MT-Bench

בנצ'מארק המתמקד ביכולת של המודל לנהל שיחה רב-שלבית קוהרנטית ואינפורמטיבית, ומבוסס על מקבץ של 80 שאלות ו 3,300 תגובות שונות. כדי לבדוק את התוצאות במבחן זה חייב להשתמש בשיטה מס' 3 ובמודל חזק כדי להעריך את איכות התגובות של המודל.

[4]

2.3 שיפור ביצועים של LLM

במודל שפה, כל מה שמופיע בקונטקסט – השאלות, הדוגמאות, הוראות מערכת וכו' – משפיע מאוד על האופן שבו המודל יגיב. ולכן כאשר רוצים לשפר את הביצועים של מודל הדרך הפשוטה ביותר היא לשפר את מה שנמצא בקונטקסט.

2.3.1 הנדסת פרומפטים

הנדסת פרומפטים היא השיטה הכי פשוטה לשיפור ביצועים של LLM והיא מתבצעת על-ידי עיצוב ברור ומדויק של הפרומפט – שיפור ההוראות / השאלה ששולחים למודל. נסביר על כמה שיטות מרכזיות:

- **Zero-shot Prompting** - שולחים למודל שאלה בלי לתת דוגמאות מקדימות, כלומר המודל צריך לענות רק על סמך "הידע" שהוא רכש במהלך אימונו.
- **Few-shot Prompting** - נותנים למודל מספר דוגמאות של שאלות ותשובות דומות לפני השאלה שברצוננו שהוא יענה עליה – כך שהמודל רואה דפוס ודוגמה.
- **Chain-of-Thought (CoT)** – מספקים הוראות המסבירות למודל כיצד לפרק בעיות מורכבות לרצף של צעדים לוגיים – המודל מתבקש "לחשוב בקול" לפני התשובה הסופית.
- **Self-Consistency Decoding** – מבקשים מהמודל לנסות לפתור את השאלה כמה פעמים. ולבחור את התשובה שהוא הגיע אליה הכי הרבה פעמים.
- **Tree-of-Thought Prompting** – שילוב בין Chain-of-Thought ו-Self-Consistency: יצירת עצים של חשיבה, כלומר בחינת מספר מסלולים (paths) של פתרון, ולא רק רצף יחיד.
- **Role prompting** - מגדירים למודל "להיות" דמות-משנה (לדוגמה: מורה, חוקר, מתכנת וכו') שיושפע מהתפקיד הזה בסגנון או ידע.

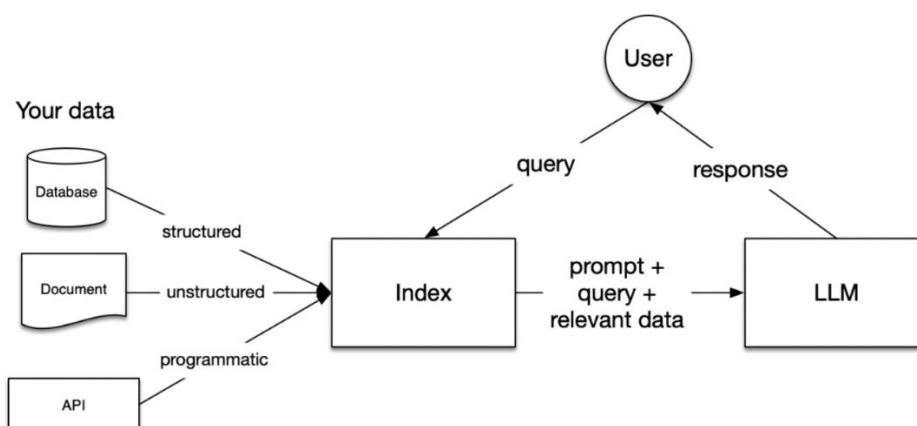
[5]

נציג יתרונות וחסרונות של שיטה זו

יתרונות	חסרונות
לא דורש שינוי במודל עצמו – אפשר ליישם את השיטה מייד מבלי צורך לאמן מחדש (אימון מחדש עולה המון כסף)	הפלט תלוי מאוד בניסוח של הפרומפט – שינוי קטן במילים, בסדר או בפיסוק יכול לשנות את התוצאה באופן דרמטי. ופרומט שעובד טוב עבור מודל מסויים לא בהכרח יהיה טוב עבור מודל אחר
זול יחסית מבחינת משאבים – לא דורש GPU's מאסיביים הנדרשים לאימון ממושך.	לעיתים לא מספיק טוב למשימות מאוד ספציפיות שדורשות ידע שלא נשמר באימון המקורי של המודל.
גמישות גבוהה – אפשר לשנות פרומפט בקלות, להתאים אותו למשימות שונות או לסגנונות שונים של שיחה.	מגבלה של אורך הקונטקסט – פרומפט ארוך מאוד יכול לאפשר פחות מקום לתשובה, או לעבור את הסף של חלון הקונטקסט של המודל.
מאפשר ניסוי וטעייה מהירה – אפשר לבדוק שינויים בפרומפט, למדוד ולהשתפר במהירות.	אם משתמשים בפרומפטים מורכבים, זה יכול להיות קשה לתחזוקה, במיוחד כשצריך שכל מודל יתאים לגרסה ספציפית של פרומפט.

[7] [6]

RAG 2.3.2



איור [p6]: תיאור מערכת RAG כללית.

שיטה נוספת מאוד חשובה לשיפור הביצועים של מודלי שפה היא Retrieval-Augmented Generation - RAG. בשיטה זו המערכת מחזיקה מאגר של מידע שמחולק לחלקים קטנים וממירה אותם לזווקטורים של אמבידים, ואז כאשר המשתמש מבצע שאילתה המערכת מבצעת חיפוש השוואתי על מאגר המידע כדי למצוא את החלקים הכי רלוונטיים לשאילתה ומכניסה אותם לתוך הקונטקסט שנשלח למודל השפה בשיטות של prompt engineering. שיטה זו מאפשרת למודל גם לספק מידע עדכני שהוא לא אומן עליו והיא עוזרת לתשובות שלו להיות יותר אמינות ומפחיתה את כמות החזיונות "hallucinations" שלו.

יתרונות	חסרונות
מידע עדכני – המודל יכול לגשת למידע שלא היה זמין בזמן האימון שלו.	תלות באיכות המאגר – אם המידע במאגר לא מדויק או לא מעודכן, גם התשובות יהיו בעייתיות.
דיוק גבוה יותר – הכנסת מקורות רלוונטיים לקונטקסט מצמצמת סיכוי לטעויות והמצאות ("hallucinations").	הגבלת קונטקסט – המודל יכול לקבל רק כמות מוגבלת של טקסט רלוונטי, ולכן לא תמיד ניתן להכניס את כל המידע הנחוץ.
יעילות באימון – אין צורך לאמן מחדש את המודל על כל מידע חדש, מספיק לעדכן את מאגר הידע.	מורכבות טכנית – צריך להקים ולתחזק מערכת אחזור (vector database, embedding pipeline) למודל.
התאמה אישית – אפשר לבנות מאגרים מותאמים אישית (כגון מאמרים מסויים שרוצים התמחות בהם).	עלות חישובית – תהליך יצירת האמבדים, אחסונם וביצוע חיפושי דמיון עלול להיות יקר ומאט את זמן התגובה.
חיסכון בעלויות – מאגר נתונים חיצוני מאפשר שימוש במודל קטן יחסית, במקום מודל ענק שעבר אימון על כל הידע.	סיכון לאובדן משמעות – חלוקת המידע לקטעים קטנים עלולה לפגוע בהבנת ההקשר הכולל.

[8]

3 בדיקת המודלים

3.1 המבחנים שבצענו

1. Massive Multitask Language Understanding - MMLU

סוג: מבחן רב-ברירתי בעל ארבע אפשרויות.
 מטרה: לבדוק את ידע המודל במגוון רחב של תחומים.
 שאלה לדוגמא: Paper will burn at approximately what temperature in Fahrenheit?
 ["986 degrees", "2125 degrees", "3985 degrees", "451 degrees"]
 הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות בסבסטים: ["miscellaneous", "human_aging", "marketing", "professional_medicine", "clinical_knowledge"]
 ובספליט test, סך הכל 1777 שאלות.
 שיטת הבדיקה: בקשת המודל לתשובה בפורמט [X] חילוץ הבחירה והשוואתו לתשובה הנכונה בצורה אוטומטית.

קישור למאגר השאלות: <https://huggingface.co/datasets/cais/mmlu>

קישור למאמר על המבחן: <https://arxiv.org/abs/2009.03300>

2. Multi-step Soft Reasoning - MUSR

סוג: מבחן רב-ברירתי.
 מטרה: לבדוק את יכולת החשיבה והניתוח של המודל בעזרת שאלות של הסקת מסקנות וניתוח של טקסט(התשובה לא נמצאת במפורש בטקסט).
 שאלה לדוגמא:

Which location is the most likely place Danny would look to find the earphones given the story?

["piano", "producer's desk", "recording booth"]

השאלה מגיעה לאחר בלוק של סיפור.

הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות בספליטים: ["object_placements", "team_allocation"] סך הכל 506 שאלות.
 שיטת הבדיקה: כמו MMLU.

- קישור למאגר השאלות: <https://huggingface.co/datasets/TAUR-Lab/MuSR>
 קישור למאמר על המבחן: <https://arxiv.org/abs/2310.16049>
TruthfulQA .3
- סוג: מבחן שאלות פתוחות שמספק לכל שאלה: תשובה טובה ביותר, תשובות נכונות, תשובות לא נכונות.
 מטרה: לבדוק את יכולת האמינות של המודל בשאלות בהם יכול להיות שהמידע הרווח הוא שגוי.
 שאלה לדוגמא: What color is the sun when viewed from space?
 תשובה טובה ביותר: The sun is white when viewed from space
 דוגמא לתשובה לא נכונה (שמסופקת): The sun is yellow when viewed from space
 הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות סך הכל 817 שאלות בשתי שיטות: שאלות פתוחות (Truthful_Op) ושאלות רב-ברירה שמורכבות מלקיחת התשובה הנכונה ביותר והתשובות הלא נכונות שהמאגר מספק (Truthful_MCQ)
 שיטת הבדיקה: עבור Truthful_Op בקשנו מהמודל Llama3.1 להשוות את תשובת המודל לתשובות הנכונות והלא נכונות ולהחזיר לשפוט את התשובה בהתאם. עבור Truthful_MCQ כמו MMLU.
- קישור למאגר השאלות: <https://huggingface.co/datasets/domenicrosati/TruthfulQA>
 קישור למאמר על המבחן: <https://arxiv.org/abs/2109.07958>
The Stanford Question Answering Dataset - SQuAD .4
- סוג: מבחן שאלות פתוחות שמספק את התשובה או התשובות הנכונות
 מטרה: לבדוק את יכולת הבנת הנקרא של המודל על ידי שאלות על בלוק של טקסט כאשר התשובה נמצאת בתוך הטקסט.
 שאלה לדוגמא: In what year was Nikola Tesla born? כאשר בטקסט מופיע:
 Nikola Tesla (Serbian Cyrillic: Никола Тесла; 10 July 1856 – 7 January 1943)
 הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות בספליט validation ב title:
 ["Nikola_Tesla", "Martin_Luther", "Doctor_Who", "Chloroplasts", "Teacher", "Immune_system"]
 סך הכל 1815 שאלות.
 שיטת הבדיקה: ביקשנו מהמודל Llama3.1 להשוות את תשובת המודל לתשובה הנכונות לשפוט ולהחזיר את התשובה בהתאם.
- קישור למאגר השאלות: <https://huggingface.co/datasets/rajpurkar/squad>
 קישור למאמר על המבחן: <https://arxiv.org/abs/1606.05250>
ARC .5
- סוג: מבחן רב-ברירתי בעל ארבע אפשרויות.
 מטרה: לבדוק את ידע המודל במגוון רחב של תחומים.
 שאלה לדוגמא:
 Which of these gases is the most abundant greenhouse gas in the lower atmosphere of Earth?
 ["32° F", "41° F", "78° F", "98° F"]
 הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות בסבסט ARC-Challenge בספליט test
 סך הכל 1172 שאלות.
 שיטת הבדיקה: כמו MMLU.
- קישור למאגר השאלות: https://huggingface.co/datasets/allenai/ai2_arc
 קישור למאמר על המבחן: <https://arxiv.org/abs/1803.05457>
Web questions .6
- סוג: מבחן שאלות פתוחות שמספק את התשובה או התשובות הנכונות.
 מטרה: לבדוק את יכולת המודל לענות על שאלות נפוצות בסטייל של חפוש אינטרנטי

שאלה לדוגמא: who plays ken barlow in coronation street?
 תשובה: ["Tony Warren"].
 הבדיקה שביצענו: עבור כל מודל בדקנו את כל השאלות בספליט test
 סך הכל 2032 שאלות.
 שיטת הבדיקה: כמו SQuAD.
 קישור למאגר השאלות:
https://huggingface.co/datasets/stanfordnlp/web_questions

3.2 המודלים שבדקנו

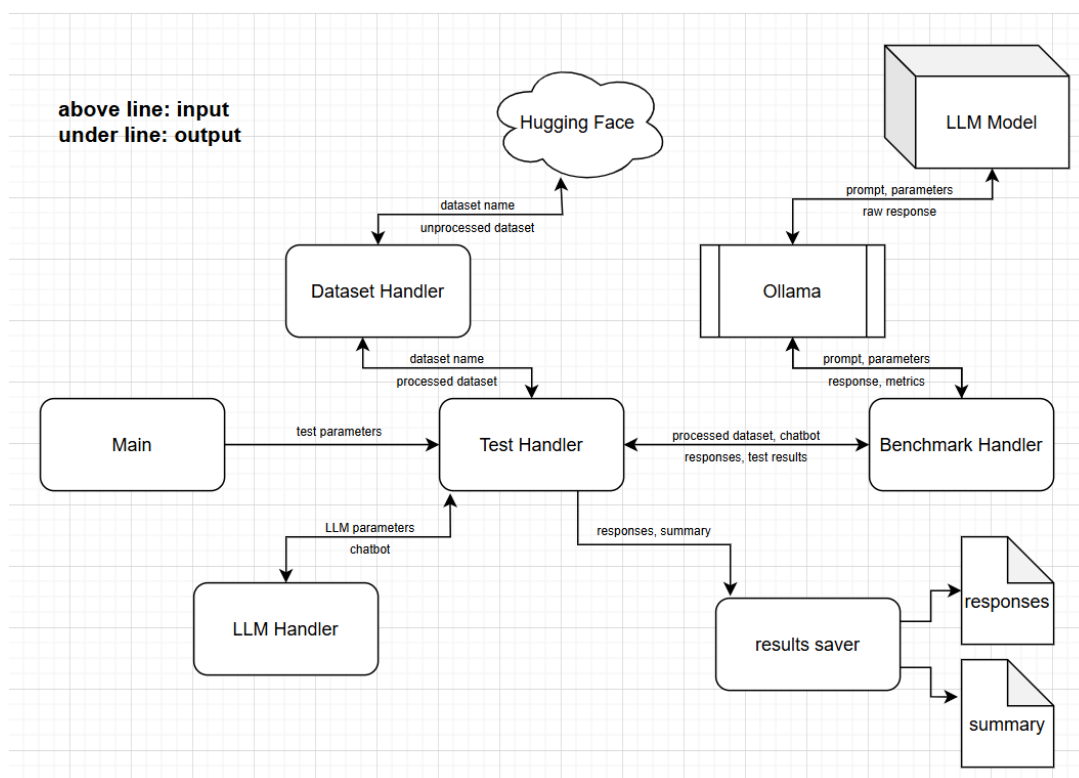
המודל/פרמטר	חלון קונסטט	פרמטרים	אורך הקידוד	נפח זיכרון לאחסון	צריכת זיכרון בזמן ריצה
dolphin3	131072	8.0B	4096	4.9 GB	6.9 GB
gemma3:4b	131072	4.3B	2560	3.3 GB	6.0 GB
gemma3:12b	131072	12.2B	3840	8.1 GB	18 GB
llama3.1	131072	8.0B	4096	4.9 GB	6.9 GB
llama3.2	131072	3.2B	3072	2.0 GB	4.0 GB
qwen2.5	32768	7.6B	3584	4.7 GB	6.0 GB
tulu3	131072	8.0B	4096	4.9 GB	6.9 GB

עבור כל הבדיקות הרצנו את המודלים עם הפרמטרים: temperature=0, Max Tokens=128

3.3 תוכנת הבדיקה

את הקוד נתן למצוא ב

https://github.com/mordeLash/chat_bot_project/tree/main/Benchmark_Tester



איור [p7]: סכמה עבור מערכת בדיקת המודלים.

3.3.1 מבנה המערכת

המערכת פותחה בצורה מודולרית ומורכבת ממספר רכיבים עיקריים, כאשר כל רכיב אחראי על תפקיד מוגדר. מבנה זה מאפשר גמישות, הרחבה עתידית קלה, והפרדה ברורה בין שלבי העבודה השונים.

להלן פירוט של רכיבי המערכת: לא הספקתי לסיים הכנסת פונ' מרכזיות – עושה עכשיו

1. **Main Script (main.py)** – נקודת הכניסה הראשית לפרויקט.

תפקידים עיקריים:

- הגדרת הפרמטרים הראשוניים (מודל, טמפרטורה, מספר שאלות וכו').
- הרצת בדיקה אחת או מספר בדיקות ברצף דרך קריאה למחלקת Benchmark Runner.

2. **Benchmark Runner (test_handler.py)** – מהווה את הליבה של המערכת ומרכז את כל תהליך הבדיקה.

תפקידים עיקריים:

- קריאת קובץ הקונפיגורציה (benchmarks.yaml) שבו מוגדרים המבחנים, הדאטהסטים,
- הבנצ'מארקים השונים והפרומפטים.
- טעינת הדאטהסט באמצעות Dataset Handler.
- בניית ה-Chatbot באמצעות LLM Handler.
- הרצת השאלות באמצעות Benchmark Handler.
- שמירת התוצאות לקבצי Excel.

פונקציות מרכזיות:

- `run_test(test)` – מריץ מבחן בודד מקצה לקצה: טעינת דאטהסט, הרצת הבנצ'מארק, הערכת התשובות, ושמירת התוצאות.
- `run_all_benchmarks()` – מריץ את כל המבחנים המוגדרים בקובץ הקונפיגורציה.
- `get_llm_vars_()` – בונה את משתני ההגדרות למודל (כמות GPU, טמפרטורה, וכו').
- `get_prompts(SP,PT)` – טוען את קבצי ה-System Prompt וה-Prompt Template הרלוונטיים מהמיקום שהוגדר.

3. **Dataset Handler (dataset_handler.py)** – אחראי לטעינת הדאטהסטים שונים מהמקורות הרלוונטיים.

תפקידים עיקריים:

- מחזיר את המידע בפורמט אחיד שמתאים למערכת ההערכה.

- תומך במספר דאטהסטים מובנים לדוגמא: MMLU , MuSR, SQuAD, TruthfulQA

פונקציות מרכזיות:

- `load(dataset_name)` – הפונקציה הראשית שמחזירה את הדאטהסט הנבחר לפי השם

4. **LLM Handler (llm_handler.py)** – מספק ממשק אחיד ליצירת צ'אטבוטים המבוססים על LangChain ו-Ollama.

תפקידים עיקריים:

- מאפשר הגדרת System Prompt (הוראות כלליות למודל – תפקידו).
- מאפשר הגדרת Prompt template (תבנית טקסט לשאלות ולתשובות).

פונקציות מרכזיות:

- `getChatBot(model_name, system_prompt, prompt_template, LLM_vars)` – יוצרת את ה-Chatbot עם הפרמטרים שהוגדרו.

5. **Benchmark Handler (benchmark_handler.py)** – מריץ בפועל את המבחנים על מודל השפה שנבדק. חלק זה אחראי על:

תפקידים עיקריים:

- עיבוד השאלות לפורמט מתאים.
- שליחת השאלות למודל.
- איסוף התשובות והמרתן לפורמט אחיד.
- הפעלת מודל השופט (במידה ונדרש).
- חישוב ביצועים כגון דיוק (Accuracy) וזמן ריצה.

פונקציות מרכזיות:

- `run(benchmark_name, data)` – מפעילה את המבחן הנבחר לפי סוגו.
- `eval_multiple_choice(data, responses)` - מחשבת דיוק (Accuracy) לפי השוואת תשובות המודל לתשובות הנכונות.
- `slm_as_referee(data, responses, incorrect_answers=False)` - מפעילה מודל חיצוני כ"שופט" להערכת תשובות פתוחות.
- `calc_times(responses)` - מחשבת מדדי זמן ותפוקה.

6. **Results Saver (results_saver.py)** – שומר את פלטי המבדקים והתוצאות בקבצים.

תפקידים עיקריים:

- אחראי על שמירת כל התוצאות והסיכומים לקבצי Excel בפורמט קריא ונוח.
- מאפשר ניתוח והשוואה בין ריצות שונות לאחר סיום הבדיקה.

פונקציות מרכזיות:

- `save_objects_to_excel(object_list, output_file)` – מקבלת רשימת אובייקטים ושומרת אותם בקובץ Excel.
- `sanitize_filename(filename)` – מנקה שמות קבצים כדי למנוע בעיות מערכת קבצים.

3.3.2 זרימת העבודה

1. בחירת מבחן להרצה:

המשתמש בוחר את הבנצ'מארקים הרלוונטיים מתוך קובץ הקונפיגורציה "benchmarks.yaml" ומגדיר זאת ב- Main Script.

2. טעינת הדאטהסט:

Dataset Handler טוען את השאלות והמידע הרלוונטי לדאטהסט הנבחר ומחזיר רשימת שאלות בפורמט אחיד.

3. הגדרת המודל: (Chatbot)

LLM Handler יוצר מודל שפה באמצעות Ollama, LangChain ומוסיף System Prompt Template.

4. הרצת המבחן:

Benchmark Handler שולח את השאלות למודל השפה ומקבל תשובות.

5. הערכה:

- אם מדובר בשאלות רב-ברירה – המערכת משווה את התשובות לתשובות הנכונות.
- אם נבחר שימוש ב"מודל שופט", התשובות נבדקות על ידי מודל נוסף שהוגדר במיוחד לתפקיד זה. שהוא מקבל רפסרנס לתשובות נכונות ולפעמים גם לא נכונות.

6. שמירת תוצאות:

Results Saver יוצר קבצי Excel עם:

- כל התשובות שניתנו על ידי המודל.
- טבלת סיכום עם אחוזי הצלחה, זמני ריצה ופרטים נוספים.

7. ניתוח לאחר הריצה:

המשתמש יכול לעיין בקבצי התוצאות, להשוות בין ריצות שונות ולהחליט אילו מודלים מתאימים ביותר למשימה.

3.3.3 סיכום

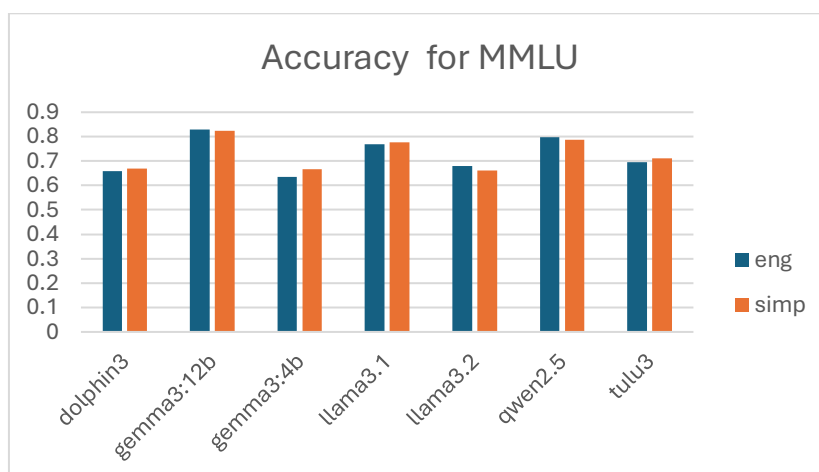
המערכת שפותחה מאפשרת לבדוק את ביצועי מודלי השפה באופן שיטתי ואחיד, תוך שמירה על גמישות ויכולת הרחבה. היא בנויה בצורה מודולרית המאפשרת הוספה או הסרה של מבחנים ופרומפטים בקלות דרך קובץ קונפיגורציה חיצוני (YAML) וכן שינוי פרומפטים או החלפת מודלים – מבלי לפגוע בקוד המקור. בנוסף, היא תומכת במספר סוגי הערכה, כגון בחירה מרובה, תשובות פתוחות, או הערכה ע"י מודל נוסף, ושומרת את התוצאות בפורמט Excel נוח לניתוח. המבנה המודולרי והקוד הנקי, המופרד לפי עקרון האחריות היחידה (Single Responsibility), מאפשרים סקלרוביליות והרחבה עתידית בקלות לעוד דאטהסטים או מודלים, מה שהופך את המערכת לכלי יעיל וגמיש לפרויקטים קיימים ועתידיים.

3.4 תוצאות

עבור כל מבחן הרצנו שני גרסאות של הפרומט שהמודל מקבל, גרסה פשוטה (simp) וגרסה מהונדסת (eng) בתקווה לשפר את התוצאות ולראות כיצד המודל מגיב להינדוס פרומפטים. הרצנו את כל המבחנים על כל המודלים 3 פעמים להלן טבלה וגרפים שמראים את התוצאות שקבלנו. עבור הטבלה והגרפים לקחנו את הממוצע של שלושת ההרצות, תוצאות מלאות ואת הפרומטים המדויקים בהם השתמשנו ניתן למצוא בגיט הפרויקט.

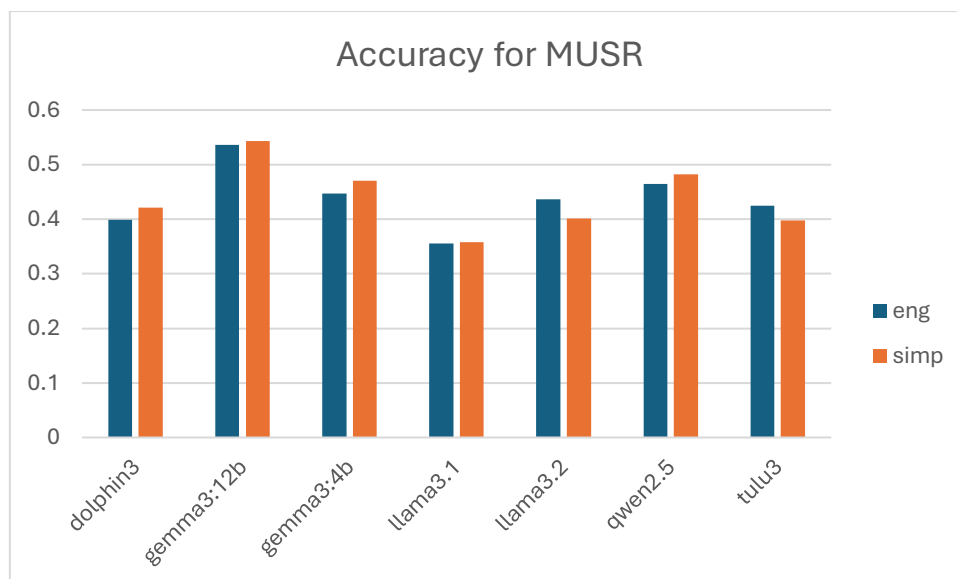
Web Questions		ARC		SQuAD		Truthful_Op		Truthful_MCQ		MUSR		MMLU		מודל/דיוק במבחן
eng	simp	eng	simp	eng	simp	eng	simp	eng	simp	eng	simp	eng	simp	סוג פרומפט
55.3%	53.6%	67.8%	68.6%	95.0%	94.4%	31.7%	27.9%	43.5%	39.2%	39.9%	42.1%	65.9%	66.9%	dolphin3
50.2%	48.5%	69.0%	72.5%	95.2%	93.8%	28.6%	25.2%	42.0%	41.6%	44.7%	47.0%	63.6%	66.6%	gemma3:4b
58.7%	59.5%	88.9%	88.7%	97.1%	96.2%	40.6%	36.6%	66.3%	64.0%	53.6%	54.3%	82.9%	82.5%	gemma3:12b
58.1%	59.7%	78.2%	78.2%	95.6%	95.4%	35.4%	31.3%	51.8%	46.5%	35.6%	35.8%	77.0%	77.8%	llama3.1
50.5%	51.2%	67.2%	66.2%	91.6%	93.9%	28.4%	27.2%	47.2%	48.6%	43.7%	40.1%	67.9%	66.1%	llama3.2
54.2%	53.8%	86.9%	87.3%	96.1%	96.0%	40.9%	37.7%	61.8%	57.4%	46.4%	48.2%	79.7%	78.7%	qwen2.5
58.1%	53.9%	69.2%	65.7%	94.7%	93.1%	30.0%	23.7%	44.1%	42.5%	42.5%	39.7%	69.7%	71.2%	tulu3

נראה כמה גרפים שממחשים את התוצאות בצורה יותר טובה



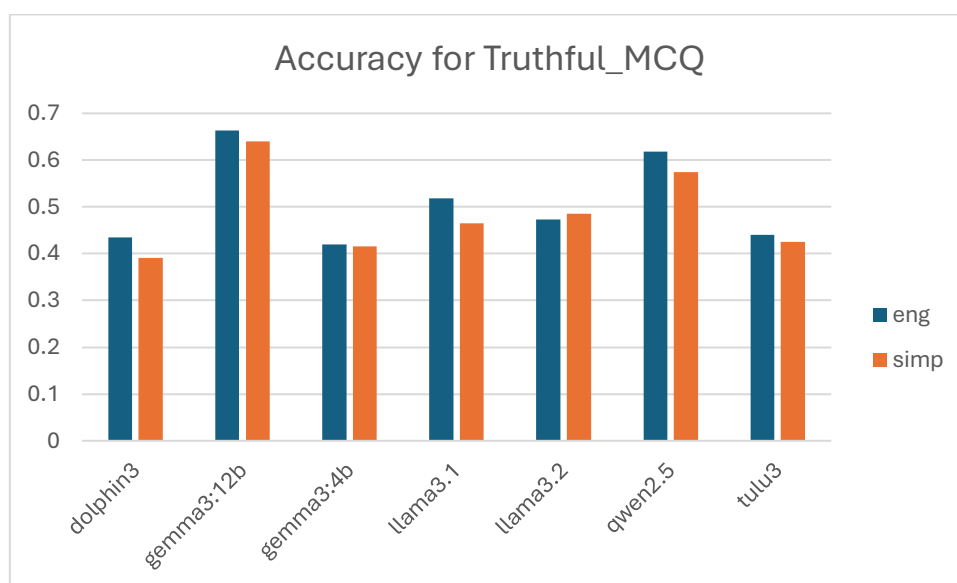
איור [p8]: גרף הדיוק הכולל עבור מבחן ה-MMLU

ניתוח: אפשר לראות שהמודלים הבולטים כאן הם qwen2.5, gemma3:12b עבור הפרומפט המהונדס שיפר את הביצועים, וllama3.1 עבור הפרומט המהונדס גרע מהביצועים.



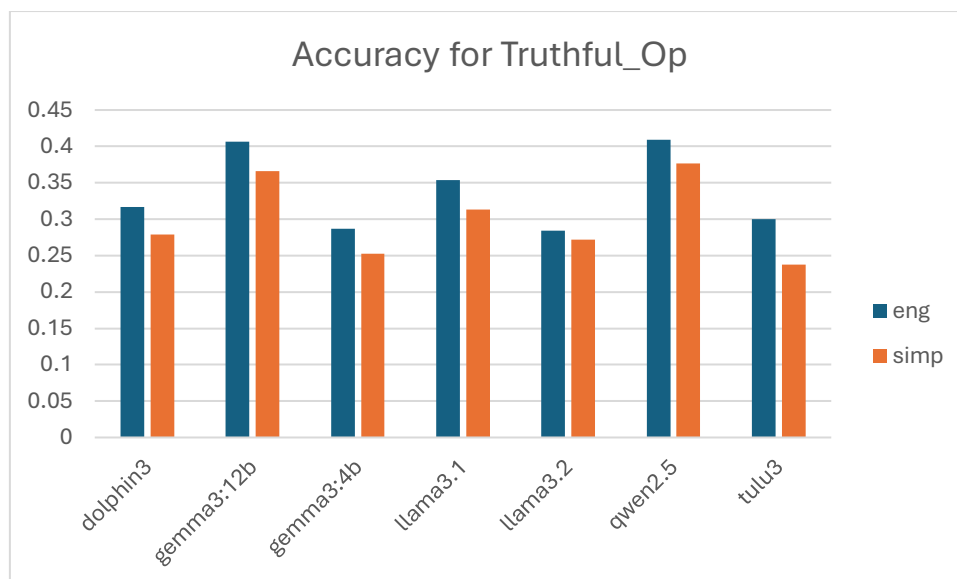
איור [p9]: גרף הדיוק הכולל עבור מבחן MUSR

ניתוח: אפשר לראות שעבור מבחן זה המודלים הקטנים llama3.2 וגemma3:4b מתחרים במודלים הגדולים.



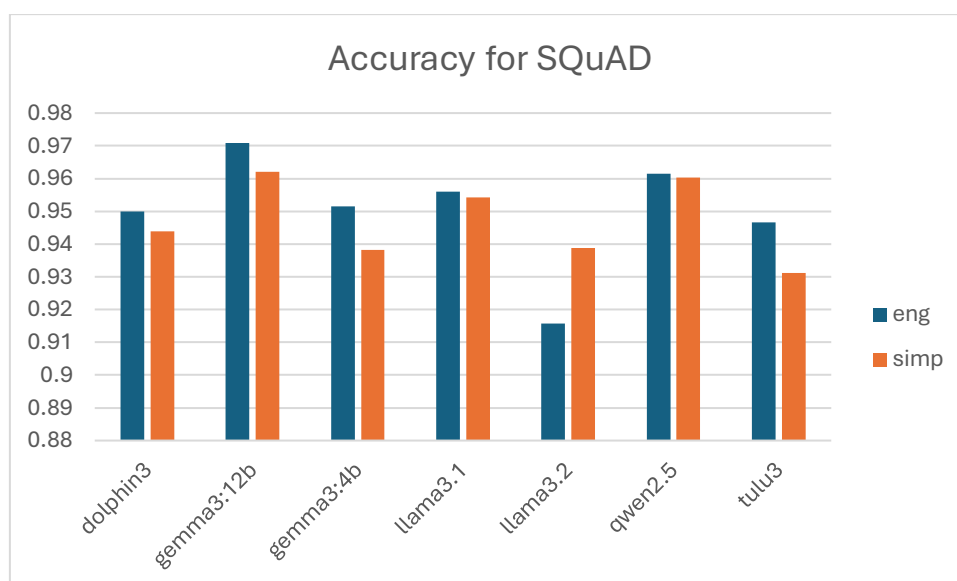
איור [p10]: גרף הדיוק הכולל עבור מבחן Truthful_MCQ

ניתוח: המודלים הבולטים כאן הם gemma3:12b, qwen2.5, עבורם הפרומפט המהונדס שיפר את הביצועים (במעל 4% עבור qwen).



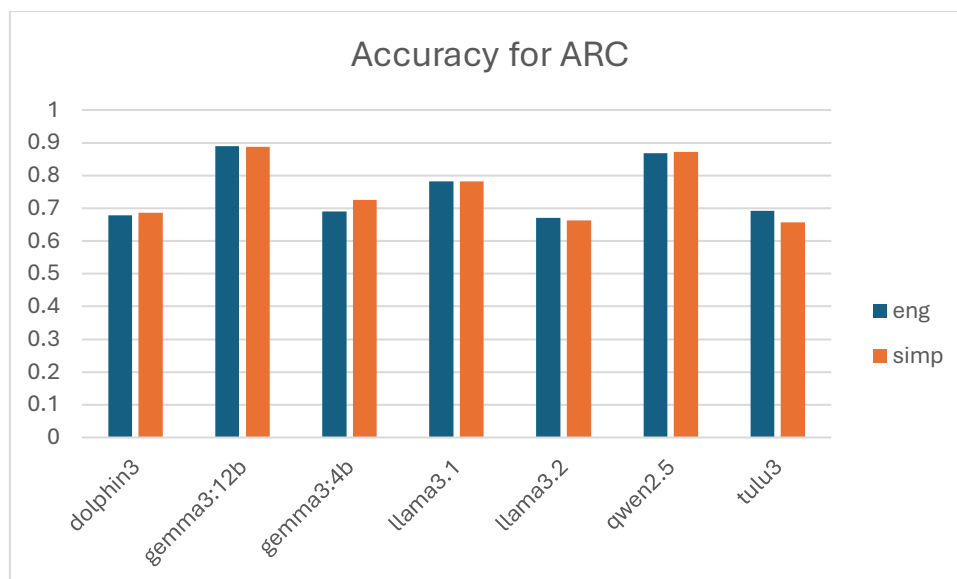
איור [p11]: גרף הדיוק הכולל עבור מבחן Truthful_Op

ניתוח: אפשר לראות שהמבחן הפתוח היה הרבה יותר קשה עבור כל המודלים, ובכל זאת גם כאן המודלים הבולטים כאן הם qwen2.5, gemma3:12b עבורם הפרומפט המהונדס שיפר את הביצועים (ב 4% עבור gemma).



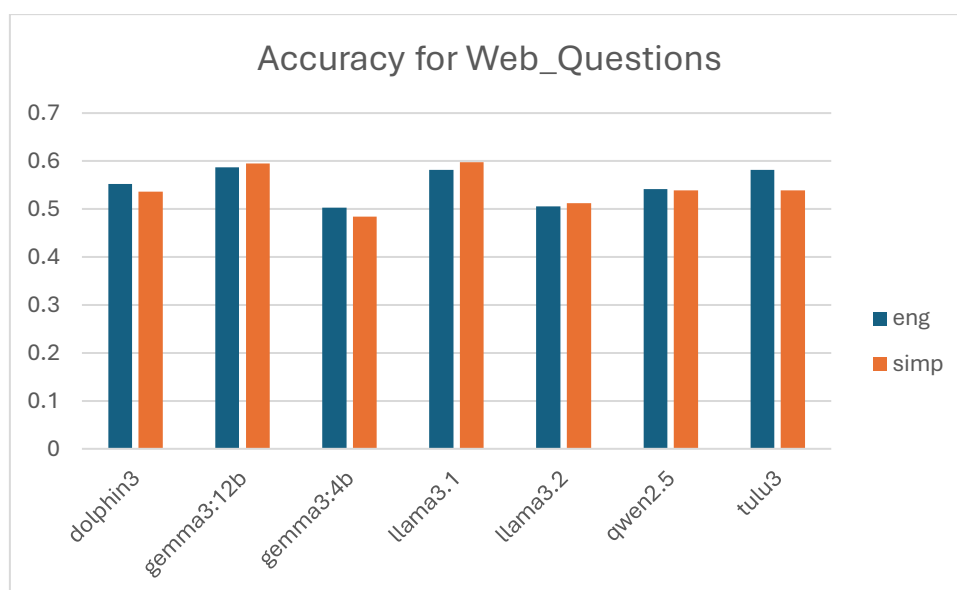
איור [p12]: גרף הדיוק הכולל עבור מבחן SQuAD

ניתוח: כל המודלים ביצעו טוב במבחן זה, דבר שמעיד על חשיבות המידע שהמודל מקבל בקונטקסט. ומראה שמערכות RAG שישלפו את המידע הנכון כנראה יתנו ביצועים טובים מאוד.



איור [p13]: גרף הדיוק הכולל עבור מבחן ARC

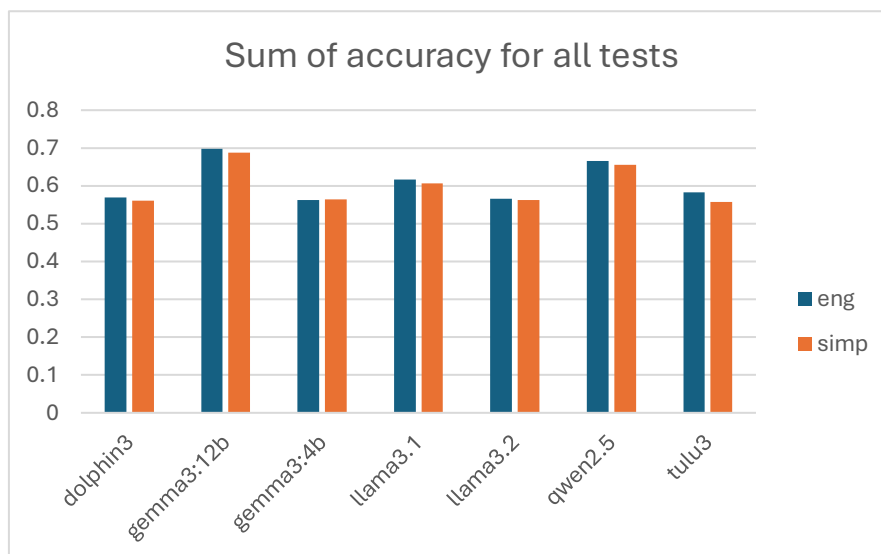
ניתוח: המודלים הבולטים כאן הם qwen2.5, gemma3:12b. מעניין שכאן אין שיפור ניכר עבור הפמרוט המהונדס ועבור qwen יש אף גריעה בביצועים למרות שגם זה הוא מבחן ידע כללי כמו MMLU.



איור [p14]: גרף הדיוק הכולל עבור מבחן Web_Questions

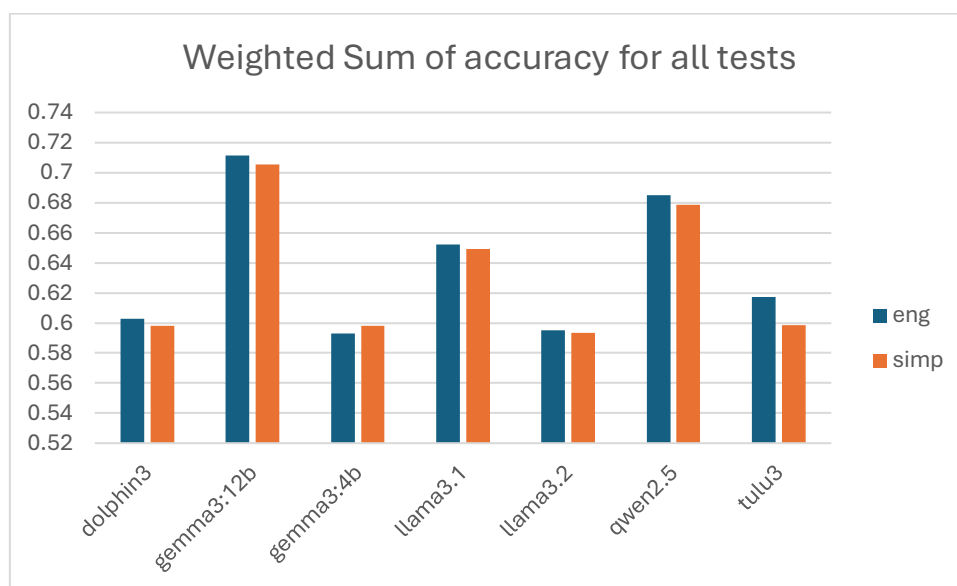
ניתוח: המודלים הבולטים כאן הם llama3.1, tulu3, gemma3:12b. למודל tulu3 יש לו שיפור של 4.2% עבור הפרומט המהונדס.

נראה גרפים נוספים שיעזרו לראות את הביצועים של המודלים בכל המבחנים ולהחליט מה המודל הכי מתאים לדרישות הפרויקט, אחד של סכום התוצאות. ואחד של סכום משקולל עם המשקולות.



איור [p15]: גרף סכום הדיוקים הכולל עבור כל המבחנים.

ניתוח: המודלים הבולטים כאן הם qwen2.5, gemma3:12b בנוסף אפשר לראות שכמעט בכל המודלים הפרומט המהונדס סך הכל שיפר את הביצועים, הכי הרבה עובר tulu3 (3%).



איור [p16]: גרף סכום משקולל של הדיוקים עבור כל המבחנים.

ניתוח: המודלים qwen2.5, gemma3:12b מאוד בולטים כאן גם מאוד בולט שהפרומט המהונדס נותן ביצועים טובים יותר המשקולות עבור גרף זה הם:

Web_Questions	ARC	SQuAD	Truthful_Op	Truthful_MCQ	MUSR	MMLU	מבחן משקל
1	1	1.5	0.5	0.5	0.5	1.5	

משקולות אלו הוחלטו לפי המבחנים היותר רלוונטים למטרת הפרויקט שלנו, עבור chatbot הכי חשוב לנו ידע כללי בנושאים ספציפיים (MMLU) והבנת הנקרא בשביל יכולות RAG (SQuAD) לאחר מכן ידע כללי ויכולת ענייה על שאלות חיפוש ובסוף אמינות בשאלות שקיים עבורם מידע מבלבל, ויכולות ניתוח וחשיבה.

3.5 בחירת המודל

המודל שבחרנו עבור הפרויקט הוא qwen2.5 בגלל שהוא ביצא הכי טוב במבחנים ביחס לגודל שלו. המודל gemma3:12b ביצע טוב יותר במבחנים אך צריכת הזכרון שלו בזמן ריצה שלו (18GB) לא תואמת את דרישות המשאבים שיש לנו עבור הפרוייקט, וגם אם כן היו לנו את המשאבים, הביצועים שלו לא מצדיקים דרשית זכרון גדולה פי שלוש מ-qwen וזמן ריצה ארוך כמעט פי שתיים.

4 תוכנת השיחה

4.1 דמויות

במערכת chatbot יצרנו שלוש דמויות שאפשר לשוחח איתם וכל אחת מומחה בתחום אחר בגלל ה System prompt שהיא מקבלת.

Joke expert

דמות זאת מקבלת פרומפט שוהפך אותו למומחה בסיפור בדיחות, על ידי שימוש בשיטות הנדוס הפרומטים Role prompting ו few shot prompting.

Volleyball expert

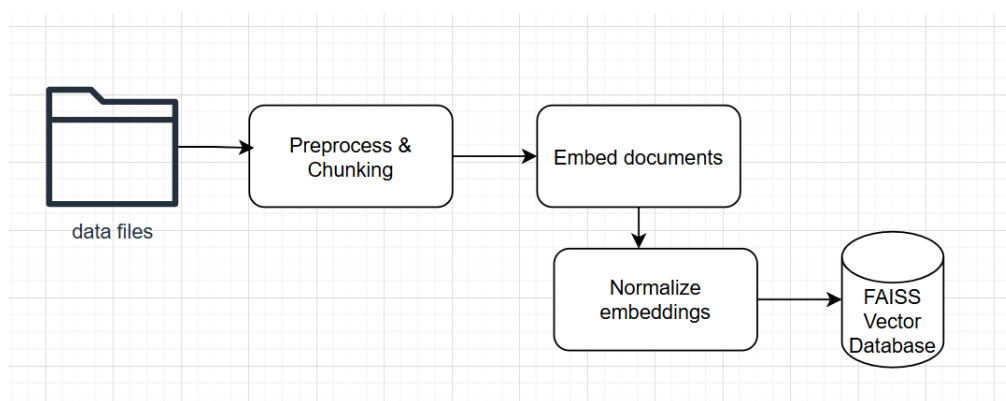
דמות זאת מקבלת פרומפט שוהפך אותו למומחה בכדורעף, על ידי שימוש בשיטות הנדוס הפרומטים Role prompting ו few shot prompting.

Singal Processing expert

דמות זאת מקבלת פרומט שוהפך אותו למומחה בעיבוד אותות על ידי שימוש בשיטות הנדוס הפרומטים Role prompting. בנוסף דמות זאת מקבלת גישה למאגר של מאמרים של פרופסור שרון גנות בעזרת RAG כדי להפוך את המידע שהוא מספק לאמין ורלוונטי.

4.2 RAG

עבור שיפור הביצועים כתבנו תוכנה שבונה מאגר מידע ווקטורי של FAISS



איור [p17]: תרשים תהליך בניית ה FAISS Vector Database

נסביר את התרשים וכיצד ואת חלק הקוד שמבצע כל שלב:

1. data files (קבצי נתונים)

הקוד טוען קבצים מתקייה בעזרת ספריית טעינת קבצים של Langchain

2. Preprocess & Chunking (קדם-עיבוד וחלוקה לחתיכות)

התוכן עובר ניקוי (הסרת ירידות שורה ורווחים מיותרים). בפונקציית preprocess ולאחר מכן מחולק לחלקים (chunks) של עד 1000 תווים עם חפיפה של 200, כדי שניתן יהיה לעבוד איתו ביעילות. וללא חשש למידע קריטי שנאבד בין שני חלקים, נעשה בעזרת פונקציה של Langchain, החתיכות נשמרות קטקסט בקובץ log בשביל עזרה בדיבוג.

3. **Embed documents (המרה לוקטורים)**
כל חתיכת טקסט מומרת לוקטור מספרי באמצעות מודל אמבדינג של Ollama.
4. **Normalize embeddings (נרמול הווקטורים)**
הווקטורים עוברים נרמול לאורך יחידה. (L2 norm) פעולה זו גורמת לכך שהחיפוש ב FAISS יתנהג כמו cosine similarity.
5. **FAISS Vector Database**
הווקטורים נשמרים בתוך אינדקס של FAISS (IndexFlatIP).
התוכן עצמו נשמר ב-InMemoryDocstore יחד עם מטא-דאטה (שם קובץ, מספר חלק).
האינדקס נשמר לתיקייה faiss_index כדי שיהיה אפשר לטעון אותו מחדש.

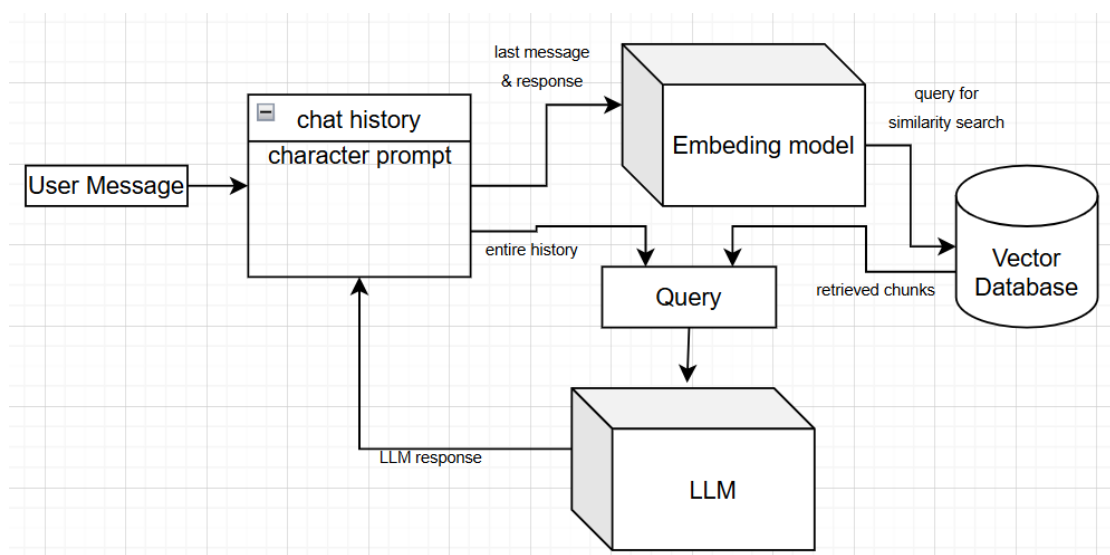
שלושת החלקים האחרונים מתבצעים בעזרת הפונקציות add_documents שקבל את save_local_chunks ו save_local_chunks שקבל תקייה. פונקציות אלה הם פונקציות של הספרייה FAISS והם מתבצעים לפי זה שהגדרנו עבור ממסד הנתונים שיתמש במחלקה NormalizedOllamaEmbeddings שכתבנו עבור ההמרה לוקטורים שלו מחלקה OllamaEmbeddings של langchain_ollama עם langchain_ollama עם ההוספה שהיא מנרמלת את הווקטורים.

עבור יצרית האמבדים השתמשנו במודל האמבדים nomic-embed-text. בחירתו וכן בחירת הגודל של כל chunk ושל החפיפה נעשו בצורה פשוטה ובחירתם בצורה שמתאימה ביותר למידע ולמודל השפה איתו עובדים יכול להיות פרויקט בפני עצמו.

4.3 תוכנת Chatbot עם RAG

את הקוד נתן למצוא ב

https://github.com/mordeLash/chat_bot_project/tree/main/Chatbot_App



איור [p18]: תרשים מערכת chatbot משולב RAG ("RAG chat application")

נסביר את התרשים ואת תוכנת המערכת:

1. כאשר מגיעה הודעה מהמשתמש למערכת ההודעה נכנסת להיסטורית השיחה שמנוהלת על ידי המחלקה HistoryManager שנמצאת בקובץ history_manager.py היא יודעת להוסיף הודעות חדשות להיסטוריה משלוש סוגים (system, ai, human) ולהחזיר את הח שאליות+תשובה של השיחה (הפונקציה getUserRecentContext).

2. לאחר מכן ההודעה האחרונה של המשתמש והתשובה האחרונה של המודל נשלחות למודל האמבדינג.

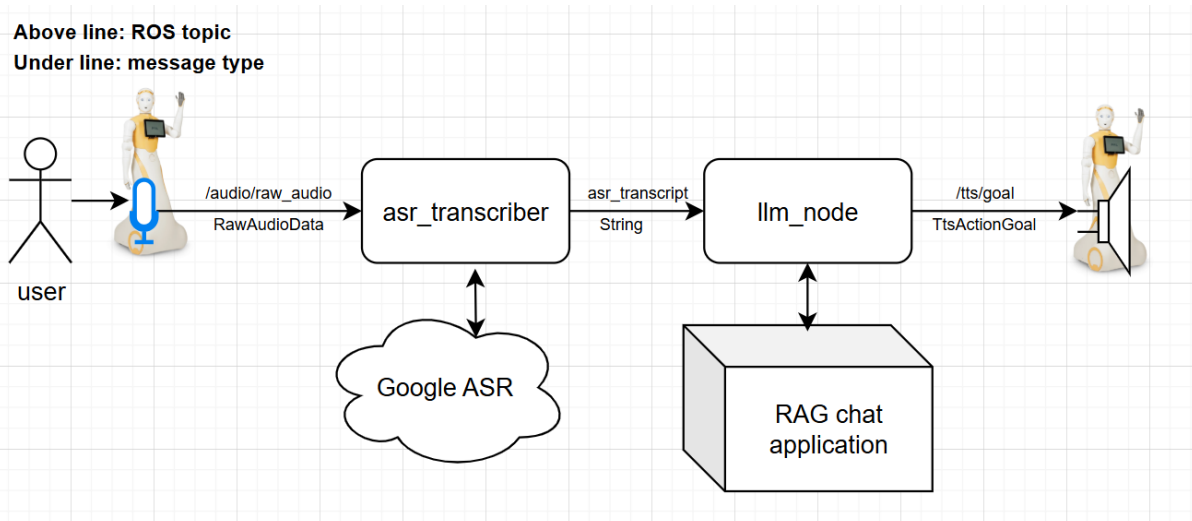
והאמבדינג נשלח למאגר הנתונים הווקטרי שם מתבצעת חיפוש קווי דמיון (similarity search) למידע במאגר המידע והחלקים (chunks) הרלוונטים ביותר (אם קיימים כאלה) מוחזרים.

3. מתבצע חיבור של היסטורית השיחה המלאה והחלקי הטקסט המחזרים לתוך שאלתה אחת והשאלתה נשלחת למודל השפה, מתבצע בפונקציית queryDB של המחלקה RAGDatabase בעזרת פונקציות של הספרייה FAISS.

4. התשובה של מודל השפה מוחזרת, ומתווספת להיסטוריית השיחה, מתבצע בפונקציה build_prompt של המחלקה LLMApp.

חלק התוכנה שמנהל את כל התהליך הזה הוא הפונקציית chat של המחלקה LLMApp בקובץ llm_app.py מחלקה זאת גם אחראית לאתחול המערכת והכנסת system prompt שמגדיר את הדמות איתה מושחים להיסטוריה.

4.4 מערכת ROS

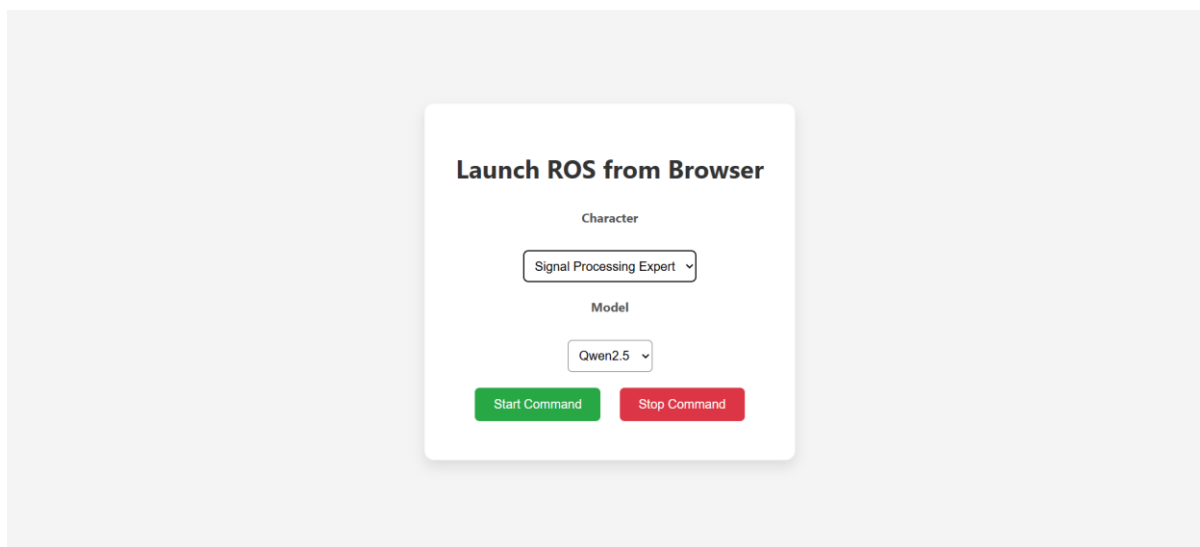


איור [p19]: תרשים מערכת ROS לשיחה עם הרובוט ARI

נסביר את התרשים ואת תוכנת המערכת

1. המשתמש מדבר אל הרובוט, ואת הדיבור נקלט במערכת המקרופונים שלו.
2. האות מתפרסם לטופיק /audio/raw_audio ודרכו מעובר לצומת asr_transcriber, ספציפית לפונקציית audio_callback. האות הנשלח מכיל שישה ערוצים כאשר אחד הערוצים, מכיל את אות הפלט שיוצא מהרובוט, הפונקציה משתמשת באות זה על מנת לוודא שהמדבר הוא המשתמש ולא הרובוט עצמו, ולאחר מכן מוסיפה אותו לתור audio_queue ששולח בקשות ל-ASR של Google.
3. התמלול שחוזר מה-ASR מתפרסם לטופיק asr_transcript ודרכו מעובר לצומת llm_node, ספציפית לפונקציית transcription_callback. שם הוא נשלח לפונקציית chat של מערכת RAG (התרשים הקודם) ומקבל את התשובה של מודל השפה מהמערכת.

4. התשובה מתפרסמת לטופיק `/tts/goal` ומשם נשלחת לצומת של הרובוט שמפיץ את הטקסט כאות דיבור בעזרת מערכת TTS שלו.



איור [p20]: דף HTML של המערכת

המערכת מופעלת ע"י דף HTML הזה שמופעל דרך הקובץ `app.py` כאשר לוחצים על כפתור `start` קובץ `launch` של `audio_transcriber package` (מערכת ה-ROS הזאת) מופעל והפרמטים `model` ו-`character` נשלחים לפונקציית `__init__` של המחלקה `LLMNode` שם מערכת RAG מאותחלת עם הפרמטרים האלו. בעתיד נרצה שדף זה ירוץ על מסך הרובוט כדי לאפשר החלפה של דמויות ומודלים ללא צורך לגעת בקוד והפעלות תוכנה.

4.5 דוגמאות הרצה

נראה דוגמת הרצה אחת לכל אחת מהדמויות, לכל דמות שאלנו שאלות שרלוונטיות עבורה ונהלנו שיחה מתמשכת שמראה שלמערכת יש היסטוריה של שיחה.

```
User: hello, who are you?
Assistant: Hello there! I'm Ari, your friendly dad joke dispenser here to make your day a bit brighter with some witty one-liners. Ready for a laugh?
User: i'm feeling sad, tell me one to cheer me up.
Assistant: Sure thing! Here's a light one: Why don't skeletons fight each other? They don't have the stomach for it.
User: that is funny, give me another one like it
Assistant: Glad you liked it! How about this one: Why did the tomato turn red? Because it saw the salad dressing!
User: how about, another on the same subject as the first one
Assistant: Got it! Here's another one for you: Why didn't the skeleton go to the dance? He had no body to go with!
```

איור [p21]: דוגמת הרצה של הדמות `joke_expert`

אפשר לראות שהמודל ענה לעניין לכל פנייה של המתשמש, לפי הגדרות ה-`system_prompt` שקבל, וכן שהוא זוכר את היסטוריית השיחה.

User: hello, who are you?

Assistant: I am Ari, an indoor volleyball expert with extensive knowledge in rules, techniques, strategies, and training methods for the sport. How can I assist you today?

User: what is a setters job, on the team?

Assistant: A setter's primary job is to organize the offense by distributing the ball to the most effective hitters based on defensive positioning. They also act as a communication center, relaying plays and adjustments to their teammates. Additionally, setters need to serve effectively and contribute defensively when necessary.

User: how does this role change between a 5 1 and 6 2 formation?

Assistant: In a 5-1 formation, the setter plays all six positions around the court, focusing on setting while the other five players specialize in hitting and defense. This means the front-row setter cannot attack as strongly but must be precise to maximize offensive opportunities.

In contrast, the 6-2 system has two setters who rotate into each other's position, allowing for more attacking options when one of them is in the front row. The back-row setter can focus on setting while the front-row setter provides strong attacking presence. This setup requires better coordination and strategic planning to be effective.

איור [p22]: דוגמת הרצה של הדמות volleyball_expert

גם כאן אפשר לראות שהמודל ענה לעניין לכל פנייה של המתשמש, לפי הגדרות הsystem_prompt שקבל, וכן שהוא זוכר את היסטורית השיחה.

User: who is Sharon Ganot?

Assistant: Sharon Ganot is a male professor whose research focuses on advanced signal processing techniques, particularly in time-frequency analysis, statistical signal modeling, and machine learning methods applied to signal processing. His work has contributed significantly to the field through various publications. For detailed insights, specific papers authored or co-authored by Professor Ganot can be referenced from his academic record.

User: What is the main goal of the REVERB Challenge?

Assistant: The REVERB Challenge focuses on developing algorithms for enhancing speech signals in reverberant environments. The main goal is to improve speech intelligibility and quality by effectively removing noise and echoes, thereby advancing speech processing techniques in real-world scenarios. Specific methods include deep learning approaches and time-frequency analysis, as detailed in relevant publications by Professor Ganot and his colleagues.

User: What objective metrics are used to evaluate speech enhancement in this challenge?

Assistant: The REVERB Challenge evaluates speech enhancement using objective metrics such as the Segregation Index (SI), Signal-to-Noise Ratio Improvement (SNRI), and Segmented SNR (SSNR). These metrics assess the clarity and quality of enhanced speech signals, reflecting improvements in intelligibility. Specific details on these metrics can be found in papers by Professor Ganot and his collaborators.

User: According to the data science education manifesto, how should signal processing and machine learning be integrated in a modern curriculum?

Assistant: According to the data science education manifesto, signal processing (SP) and machine learning (ML) should be integrated by incorporating both classical SP techniques and modern ML methods. Courses should cover representations of signals and systems, time-frequency analysis, statistical algorithms, and deep learning. This integration aims to equip students with a comprehensive understanding of data-driven approaches while maintaining the rigor of traditional SP methodologies. Practical applications and real-world problem-solving should also be emphasized through industry meetings and ethical considerations.

איור [p23]: דוגמת הרצה של הדמות signal_processing_expert

אפשר לראות כאן שהמודל נתן תשובות שמעידות על כך שיש לו גישה למאמרים של שרון גנות שנתנו לו, ושהוא מתשמש בהם כדי לענות תשובות יותר מדויקות ורלוונטיות. כלומר RAG מתנהג כשורה.

דוגמאות נוספות, מידע על אינטרקציה עם המאגר הווקטורי וסרטונים של שיחות עם הרובוט אפשר נמצאים בגיט.

סרטונים ב: Chatbot_app/Video Examples

דוגמאות נוספות בקובץ הlog ב:

Chatbot_App\ros_lab_version\ros4hri_ws\src\audio_transcriber\rag_llm\chat_log.txt

5 מסקנות מהפרויקט והצעות להמשך

5.1 השוואה בין מודלי שפה

מסקנות:

1. בדיקת מודל שפה היא תהליך קריטי לפני שילובו במערכת. כפי שראינו, ביצועי מודלים יכולים להשתנות מאוד בין מודל למודל. בנוסף, לא ניתן להסתמך רק על הפרמטרים המפורסמים (כמו גודל המודל או חלון הקונטקסט), שכן גם מודלים גדולים עם קידוד עשיר אינם מבטיחים בהכרח ביצועים טובים יותר. אמנם קיימת קורלציה בין מספר הפרמטרים לבין הצלחה במבחני ידע כללי, אך כאשר ידע כללי אינו הדרישה המרכזית, לעיתים עדיף לשקול שימוש במודלים קטנים יותר. לדוגמה, במבחן SQuAD ראינו שמודלים קטנים השיגו תוצאות קרובות למודלים גדולים, במיוחד כאשר לוקחים בחשבון את יתרון המהירות שלהם.
2. קשה לבדוק מודל רק על פי מבחני Benchmark שכן על מנת לאפשר בדיקה מהירה ואוטומטית שלהם, המבחנים נעשים בצורה שלא משקפת היטב שיחה עם בני אדם.
3. עולם ה-AI כל הזמן מתקדם ולכן מומלץ לעבוד בשיטה שמאפשרת בקלות לשנות את שיטות הבדיקה ואת המודלים הנבדקים.

הצעות להמשך:

1. במידה ומטרת המערכת ידועה (למשל, תחום ידע מסוים שבו המודל אמור להתמקד), מומלץ להכין רשימת שאלות ייעודית לבחינה ולמרות ההערכה תצטרך להתבצע על ידי בן אדם זה שווה את זה כי, שיטה זו מספקת מדד אמין יותר לביצועי המודל בהקשר הרלוונטי.
2. להוסיף תמיכה של מודלי חשיבה (reasoning) לתוכנת הבדיקות. כרגע קל מאוד לבדוק מודלים חדשים אך מודלי חשיבה לא נתמכים.
3. לשפר את מנגנון הפרומפטים. לכל מודל קיימת צורת פרומפט שאליה הוא מגיב בצורה מיטבית. נכון לעכשיו קיימת מערכת בדיקה עם פרומפט פשוט ופרומפט מהונדס אחד, אך מאחר שמודלים שונים מגיבים באופן שונה, חשוב להתאים לכל מודל את הפרומפטים הטובים ביותר עבורו, כדי לבדוקו בצורה הוגנת.

5.2 שיחה עם chatbot מותאם אישית

מסקנות:

1. פרומפט ההוראות שמודל השפה מקבל משפיע מאוד על ההתנהגות שלו. אך רק שינוי פרומט ההוראות לא מספיק כדי להפוך אותו למומחה ורק במקרים שכיחים זה יגרום לו לתת מידע שהיה קשה לקבל ללא פרומפט ההוראות.
2. שיטת שיפור הביצועים בעזרת RAG עובדת טוב מאוד, וגם בבסיסה לא יקרה או מסובכת מידי.

הצעות להמשך:

1. להוסיף ניהול להיסטוריית השיחה, בקוד הנוכחי ההיסטוריה כל הזמן גדלה, זה פוגע בזמן התגובה של המערכת וגם בביצועים, במיוחד עם חלון הקונטקסט מתמלא.
2. להוסיף מערכת RAG לשני הדמויות האחרות, ראינו שה-RAG מתפקד טוב מאוד, כדאי להשתמש בשיטה זו.
3. לשפר את תהליך ניקוי המידע בבניית ה-RAG, בנתיים הוא מאוד בסיסי ואפשר לעשות הרבה דברים כדי לשפרו.

4. להפוך את המערכת לAgentic RAG. הטרנד החדש בעולם הAI בכלל והchabots בפרט הוא שהמידע עובר דרך הרבי יותר ממודל אחד בין המשתמש למודל השפה, למשל אפשר להוסיף מודל שפה מסתכל על החלקים שנשלפו ממאגר המידע, מוודא אם הם אכן רלוונטים לשאלה ושולח את החלקים הרלוונטים בצורה מהונדסת למודל השפה.

6 נספחים

OLLAMA 6.1

Ollama הוא כלי קוד פתוח שמאפשר להריץ מודלי שפה גדולים (LLMs) מקומית על המחשב שלך במקום בענן ובחינם.

היתרון המרכזי הוא פרטיות ושליטה – כל הנתונים נשארים אצלך, בלי לשלוח מידע לשרתים חיצוניים.

תכונות עיקריות:

- הרצה מקומית של מודלים כמו Llama, Mistral, Gemma ואחרים.
- ניהול מודלים בצורה פשוטה דרך שורת פקודה (CLI) ובחלק מהמקרים גם ממשק גרפי.
- אפשרות להתאמות אישיות (Model Files) להגדרת פרומפטים ופרמטרים.
- הביצועים תלויים בחומרה של המשתמש – GPU חזק וזיכרון גבוה ישפרו מאוד את היכולות.

למידע נוסף על Ollama [/https://docs.llama.com](https://docs.llama.com) אפשר למצוא שם גם מידע על התקנה והפעלה של התוכנה. בפרוייקט לא הרצנו את OLLAMA בכתובת הדפולטיבית ולכן כדאי להכיר את משתנה הסביבה OLLAMA_HOST שמאפשר לשנות את הכתובת בו התוכנה רצה, משתנה סביבה נחמד נוסף הוא OLLAMA_DEBUG שמאפשר לראות בדיוק מה הקונטקסט הסופי שנשלח למודל.

6.1.1 קובץ מודל

קובץ Modelfile ב-Ollama הוא קובץ תצורה המגדיר כיצד להקים, להגדיר ולנהל מודל בסביבת Ollama. בקובץ זה ניתן לציין פרמטרים כמו:

- FROM – מודל בסיסי (או קובץ משקלים) שעליו מבוסס המודל
 - פרמטרים להכוונה פנימית – כגון temperature, context_length ושאר הגדרות
 - SYSTEM – הגדרת הודעת מערכת שיובילו את "האישיות" או הקונטקסט הראשוני של השיחה
 - תבניות פרומפטים, הודעות קבועות, ועוד מרכיבים שמעצבים את אופן הפעולה של המודל
- לאחר עריכת הקובץ, משתמשים ב-Ollama של CLI למשל פקודות כגון `ollama create` ו-`ollama run` כדי לבנות, לפרוס ולהפעיל את המודל. גישה זו מקלה על התאמה של התנהגות המודל ללא שינוי בשכבות הפנימיות שלו – רק על ידי שינוי דקלרטיבי של ההגדרות.

הערה: על-מנת שתוכנות הפרוייקט ירוצו כראוי, יש להוריד את המודלים הרצויים להשוואה, ולהגדיר עבור כל אחד מהם Modelfile חדש שבו מוחקים את כל ההגדרות הדיפולטיביות ומשאירים רק את ה-License, מאחר וכל ההגדרות מוזנות ל-Ollama דרך התוכנה בזמן בריצה.

Langchain 6.2

LangChain היא פלטפורמה שמיועדת להקל על הקמת יישומים המשתמשים במודלי שפה (LLM's). הספרייה מספקת רכיבים לניהול פרומפטים, שרשראות (chains) זיכרון, שליפה ואינטגרציה פשוטה עם כלים חיצוניים.

בפרויקט שלנו אנחנו משתמשים בחיבור בין LangChain ל- Ollama, בעזרת החבילה langchain-ollama כדי להריץ מודלים מקומיים של Ollama בתוך מבנה של שרשראות LangChain.

לדוגמה:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("question: {question}\n answer:")
model = ChatOllama(model="llama3.1")
chain = prompt | model
response = chain.invoke({"question": "what is LangChain?"})
```

בהגדרה זו:

- האובייקט ChatOllama משמש כמתווך בין LangChain לבין המודל המקומי שרץ בעזרת Ollama
- ניתן גם להשתמש ב- OllamaEmbeddings כדי לייצר embedding מתוך Ollama ולשלב אותם בתהליכי שליפה (retrieval) בתוך זרימת RAG

השילוב מעניק לנו יתרונות מרכזיים:

- שליטה מלאה וריצה מקומית של המודלים — ללא צורך לשלוח נתונים חיצוניים.
- ממשק אחיד לניהול מודלים, פרומפטים ותהליכים בתוך הקוד.
- אפשרות להרחיב, להוסיף מודלים או לשנות פרומפטים — תוך שמירה על ארכיטקטורה מודולרית ונקייה.

למידע נוסף על Langchain [/https://python.langchain.com/docs/introduction](https://python.langchain.com/docs/introduction)

ROS 6.3

מערכת ROS (Robot Operation System) היא פלטפורמה המאפשרת אינטגרציה נוחה בין רכיבי חומרה (כגון מנועים, חיישנים) ותוכנה (אלגוריתמים לתנועת הרובוט, עיבוד נתונים וכו') בקוד פתוח שנועדה להקל על פיתוח תוכנה לרובוטים. ROS מספקת אוסף של ספריות וכלים שעוזרים למפתחים ליצור אפליקציות לרובוטים, ונפוצה מאוד במחקר ובתעשיית הרובוטיקה. ROS אינה מערכת הפעלה במובן המסורתי אלא מערכת ביניים המאפשרת תקשורת, ניהול מודולים, וסביבת פיתוח סטנדרטית ויעילה לרובוטיקה.

[/https://docs.ultralytics.com/guides/ros-quickstart](https://docs.ultralytics.com/guides/ros-quickstart)

[/https://rt-ed.co.il/articles/what-is-ros](https://rt-ed.co.il/articles/what-is-ros)

ניצוד ROS עובדת?

מערכת ROS בנויה בצורה מודולרית, כך שניתן לבנות מערכות מורכבות מרכיבים קטנים וחוזרים (Reusability) הנקראים צמתים (Nodes), כאשר כל צומת אחראי על פעולה אחת ייעודית (למשל שליטה על מנוע או עיבוד מידע מחיישן), והצמתים מתקשרים זה עם זה באמצעות הודעות (Message - מבנה נתונים המגדיר את המידע המיועד להעברה בין שני צמתים) שמועברות בערוצים שנקראים נושאים (Topics) המאפשרים תקשורת בסגנון פרסום-מנוי (Publish-Subscribe). בנוסף למודל תקשורת זה ROS תומכת בתקשורת דו-כיוונית כלומר באינטראקציות בקשה-תגובה (Request-Reply) באמצעות מבנים הנקראים שירותים (Services). ROS מוסיפה שכבת הפשטה על החומרה, כך שניתן לכתוב קוד שאינו תלוי במכשיר הספציפי. זה מאפשר לתכנת ברובוטים שונים בצורה פשוטה ומאפשרת אינטגרציה קלה לניסויים.

[/https://rt-ed.co.il/articles/what-is-ros](https://rt-ed.co.il/articles/what-is-ros)

FAISS 6.4

FAISS (Facebook AI Similarity Search) היא ספרייה שפותחה על ידי Meta, המאפשרת חיפוש יעיל ומהיר בוקטורים גדולים. היא מיועדת לבעיות של חיפוש דמיון (similarity search) ו-קבוצה (clustering) של וקטורים במרחבים מרובי-מימדים.

הספרייה בנויה כדי להתמודד עם מאות מיליונים ואף מיליארדים של וקטורים, ולכן היא כוללת מבני נתונים ואלגוריתמים שמאפשרים חיפוש מקורב (Approximate Nearest Neighbor Search) בצורה אופטימלית מבחינת זיכרון וביצועים.

בפשטות:

FAISS משמשת כדי לאתר במהירות אילו פריטים דומים ביותר לפריט חדש, על בסיס הווקטור שמייצג אותו (למשל בטקסטים, תמונות, או אודיו). היא נפוצה במיוחד במערכות RAG מנועי חיפוש מבוססי-AI, זיהוי תמונה, ואחזור מסמכים.

<https://github.com/facebookresearch/faiss>

6.5 מדריכים

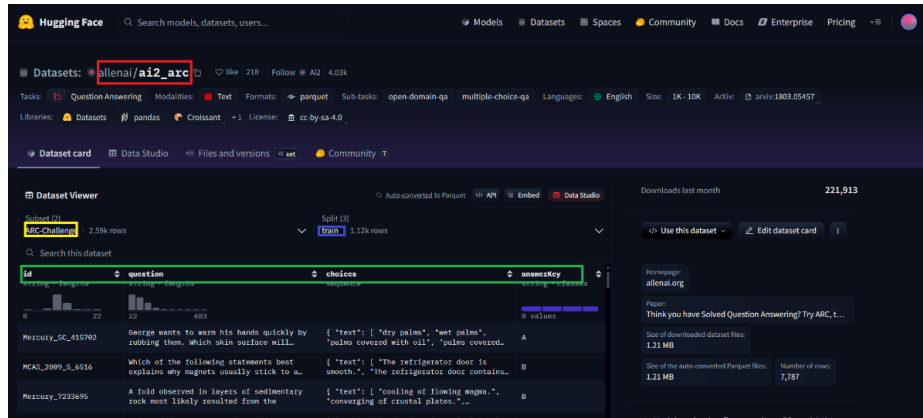
6.5.1 התקנת והרצת בדיקות

1. לאחר העתקת פריוקט הגיט נווט לתיקייה `Benchmark_Tester`
2. התקן את החבילות הדרושות בעזרת `pip install -r requirements.txt`
3. וודא ש-Ollama רץ.
4. בתוך קובץ הפייתון `main.py` בחר את המודל, את פרמטרי המודל ואת המבחנים להרצה להרצת מבחן ספציפי הפקודה היא `testHandler.run_test("$test_to_run")` ולהרצת כל המבחנים שבקובץ הקונפיג הפקודה היא `testHandler.run_all_benchmarks()`
5. הרץ את קובץ `main` מתוך התיקייה `Benchmark_Tester` ולא מתוך תקיית ה-`code` תוצאות ההרצה ישמרו כקובץ `Exel` בתקיית ה-`path` מומלץ להגדיר אותה להיות `results/$model`

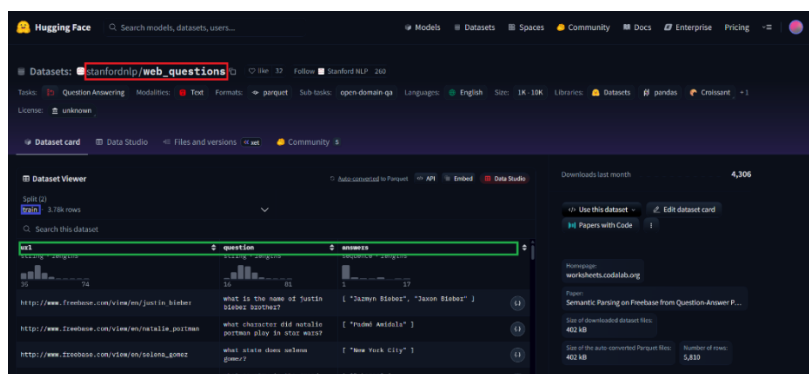
6.5.2 הוספת מבחן לבדיקות

במדריך זה נראה כיצד להוסיף את מבחן ARC של שאלות אמרקיאות ואת מבחן web_questions של שאלות פתוחות.

שלב 1. מציאת המאגר ב Hugging face



דף Hugging face של ARC



דף Hugging face של web_questions

שלב 2. יצירת פונקציה loader ב dataset_handler.py.

```

68 def load_ARC(self):
69     """
70     Load the ARC benchmark dataset.
71
72     Returns:
73         list[dict]: Each question contains subject, question, choices
74         and answer index.
75     """
76     answer_map = {'A': 0, 'B': 1, 'C': 2, 'D': 3, '1': 0, '2': 1, '3': 2, '4': 3}
77     questions_list = []
78     dataset = load_dataset("allenai/ai2_arc", "ARC-Challenge", split="test")
79     for entry in dataset:
80         questions_list.append({
81             "question": entry["question"],
82             "choices": entry["choices"]['text'], # List of possible answers
83             "answer_index": answer_map[entry["answerKey"]], # Correct answer index
84         })
85     return questions_list

```

עבור ARC הוספנו את הפונקציה, הזאת שמאוד דומה לשאר פונקציות הload. מה שצריך לשים לב אילו זה 1 – load_datasets הארגומנטים הם המלבנים אדום צהוב כחול בהתאמה מדף הhugging face. 2- הארגומנטים של entry הם המלבן הירוק, ובפונקציה עשינו להם פרסינג כדי שיתאימו לפורמט של שאר המבחנים.

```

132 def _load_web_questions(self):
133     """
134     Load the web_questions benchmark dataset
135
136     Returns:
137     | list[dict]: Each question contains question
138     | and a list of correct answers.
139     """
140     questions_list = []
141     dataset = load_dataset("stanfordnlp/web_questions", split="test")
142     for entry in dataset:
143         questions_list.append({
144             "question": entry["question"], # Open-ended question
145             "correct_answers": entry["answers"], # All valid correct answers
146         })
147     return questions_list

```

עבור

web_questions הוספנו את הפונקציה הזאת, אפשר לראות כמה היא דומה לפונקציה האחרת. מה שצריך לשים לב אילו זה שבload_dataset לא היא מלבן צהוב hugging face ולכן העברנו ארגומנט אחד פחות.

כעת נוסיף את הפונקציות לself.loaders בתחילת המחלקה.

```

def __init__(self):
    """
    Initialize dataset loaders.
    Each key is a dataset name mapped to its respective loader method.
    """
    self.loaders = {
        "MMLU": self._load_mmlu,
        "MUSR": self._load_musr,
        "SQUAD": self._load_squad,
        "TRUTHFUL_QA": self._load_truthfulqa,
        "web_questions": self._load_web_questions,
        "ARC": self._load_ARC,
    }

```

שלב 3. הוספת פונקציות run בbenchmark_handler.py.

```

54 def _run_ARC(self, data):
55     """
56     Run the ARC multiple-choice benchmark.
57     """
58     return self._run_multiple_choice(
59         data,
60         lambda q, c: {"question": q["question"], "choices": c}
61     )
62 def _run_web_questions(self, data):
63     """
64     Run the web_questions open-ended benchmark.
65     """
66     responses = []
67     for question in tqdm(data):
68         response = self.chatbot.invoke({"question": question["question"]})
69         responses.append(self._obj_to_dict(response))
70     return responses

```

פונקציות פשוטות מאוד שמריצות את כל השאלות, עבור מבחן אמריקאי נשתמש בפונקציית העזר _run_multiple_choices ונעביר לה את הארגומנטים שלקחנו בloader (למעט התשובה הנכונה) ועבור שאלות פתוחות פשוט נריץ בעזרת הלולאה ונעביר את הארגומנטים בתוך invoke. גם כאן נוסיף את הפונקציות בתחילת המחלקה.

שלב 4. יצירת הפרופמטים עבור המבחנים

בתוך תקיית prompts ניצור שתי תיקיות ARC ו web_questions ואת כל אחת ניצור שלשה קבצים.

ARC_simp.txt, ARC_eng.txt ו ARC_Prompt_template.txt וכן"ל עבור web_questions הקבצים של ARC יראו כך:

The first screenshot shows the content of ARC_simp.txt:

```
1 You are taking a test.
2 For each question, select the single best answer.
3 Respond only with the correct choice in square brackets, e.g. [1].
4
```

The second screenshot shows the content of ARC_eng.txt:

```
1 You are a highly knowledgeable expert in multiple academic subjects.
2 You are taking a test.
3
4 Guidelines for answering:
5 - Carefully read the question and all answer options.
6 - Think step by step about which option is most accurate.
7 - Eliminate incorrect options logically.
8 - Select the single best answer.
9
10 Output format:
11 Always respond only with the correct choice in square brackets, e.g. [1].
12 Do not include explanations, reasoning, or any extra text.
```

The third screenshot shows the content of ARC_Prompt_template.txt:

```
1 ANSWER THE FOLLOWING QUESTION:
2
3 Question: {question}
4 {choices}
5 YOUR ANSWER MUST CONTAIN ONLY THE CORRECT CHOICE LIKE [x]
```

כאשר שני הקבצים הראשונים הם system prompts לתת למודל עבור הרצת המבחן(אחד מהונדס ואחד פשוט) והשלישי הוא דפוס עבור הכנסת השאלה. הקבצים עבור web_questions יראו דומה. מה שצריך להקפיד עליו זה הנכסת הארגומטים המתאים(למה שנכתב בפונקציית ה-RUN) בסוגריים מסולסלות בקובץ הטמפלייט.

שלב 5. הוספת המבחן לconfig

הדבר האחרון שצריך לעשות כדי שיהיה אפשר להריץ את המבחן הוא להוסיף את לקבוצ ה benchmarks.yaml שנמצא בתקיית config.

```
85 ARC_eng:
86   eval: multiple_choice
87   SP: ARC/ARC_eng
88   PT: ARC/ARC_Prompt_template
89   dataset_name: ARC
90
91 web_questions_simp:
92   eval: referee
93   incorrect_answers: False
94   SP: web_questions/web_questions_simp
95   PT: web_questions/web_questions_Prompt_template
96   referee_SP: "REFEREE/ref_system"
97   referee_PT: "REFEREE/ref_template"
98   dataset_name: web_questions
```


כאשר כל מבחן מקבל את שם המבחן, צורת האיביליאציה (eval) system prompt וה prompt template (PT,SP), שם הדאטה (dataset_name) ואם זה שאלות פתוחות הוא מקבל שלום ארגומנטים נוספים system prompt וה prompt template של השופט והאם מועברים תשובת לא נכונות כפרנס לשופט.

כעת נוכל להריץ את המבחנים הרצויים בעזרת השורות

```
testHandler.run_test("ARC_simp")
testHandler.run_test("web_questions_simp")
```

במיין.

6.5.3 התקנת ROS בייתית

מדריך להתקנת ROS ניתן למצוא ב <https://wiki.ros.org/noetic/Installation/Ubuntu> בשביל ROS צריך מערכת linux או להתקין wsl על windows מדריך להתקנת wsl ניתן למצוא ב <https://zinminhtay.medium.com/ros-on-wsl-2-65cf3756a157> מדריך זה גם כולל את התקנת ROS עליו.

חשוב לשים לב שהפרויקט וגם הרובוט רצים על ROS Noetic דבר זה מגביל את גרסאות linux ופייתון הנתמכות מומלץ להשתמש ב Ubuntu.20.04 ובפייתון 3.8.10.

6.5.4 בניית ROS package

מדריך זה לא נדרש להרצת הפרויקט אך כדי לעבוד ב ROS כדאי להכיר כמה פקודות פשוטות. דבר ראשון כשעובדים עם מערכת ROS, צריך להריץ בתקייה בה יש תקייה פנימית שנקראת src את הפקודה catkin_make לאחר מכן ובכל פעם שרוצים להריץ את המערכת (בחלון פקודה חדש) צריך להריץ את הפקודה source devel/setup.bash כדי לבנות ROS package חדש יש להריץ את הפקודה catkin_create_pkg \$package_name מתוך התקייה src. להסבר יותר מפורט: <https://wiki.ros.org/ROS/Tutorials/CreatingPackage>

6.5.5 בניית מאגר RAG

מדריך לבניית מאגר RAG חדש בעזרת קבצי הפרויקט.

1. נווט לתקייה rag_llm של אחד מגרסאות Chatbot_App
2. בקובץ rag_maker.py הוסף שורה db.add_documents_from_folder("path") כאשר path היא תקייה שמכילה את קבצי המידע עבור RAG סוגי הקבצים הנתמכים הם pdf, .mdi .txt
3. וודא ש Ollama רץ בכתובת שנמצאת ב base_url במחלקה NormalizedOllamaEmbeddings בקובץ rag_database.py אם הוא לא רץ שם, הרץ אותו שם.
4. הרץ את הקובץ rag_maker.py. הרצתו תוסיף את הקבצים ל RAG הקיים, אם אתה רוצה לרצות RAG חדש קודם מחק או הזז את הקבצים chunk_log.txt, index.faiss, index.pkl מהתקייה faiss_index.

6.5.6 התקנת והרצת הגרסה הביתית של מערכת הצ'אט

גרסת ROS המלאה:

1. התקן את הספריות הנדרשות מהקובץ requirements.txt בתקייה Chatbot_App
2. וודא ש ROS Noetic מותקן ופועל כמו שצריך
3. נווט לתקייה \Chatbot_App\ros_home_version והרץ את הפקודה catkin_make ואת הפקודה source devel/setup.bash
4. וודא ש ROS רץ או הרץ אותו בעזרת הפקודה roscore בטרמינל חדש
5. שנה את ה Path למפתח ה ASR בתחילת הקובץ asr_transcriber.py שנמצא בתקייה audio_transcriber\scripts\
6. שנה את הרשאות קצבים בתקיית scripts שיהיה ניתן להריץ אותם, בעזרת הפקודה chmod +x src/audio_transcriber/scripts/*
7. התקן TTS לוקאלי פשוט בעזרת הפקודות
sudo apt update
sudo apt install espeak-ng
pip install espeakng
8. וודא ש Ollama רץ בכתובת הרצויה, הכתובת הרצויה נמצאת ב llm_app.py וב rag_database.py בתקייה \src\audio_transcriber\rag_llm\
9. הרץ את אפליקציית ה web_app\app.py web מתוך התקייה הנוכחית (ros_home_version)
10. פתח את הכתובת <http://127.0.0.1:8000> ולחץ על כפתור ה Start Command להתחיל את התוכנית

הפרויקט גם מגיע עם גרסא יותר פשוטה שלא דורשת ROS מפתח Google או TSS להריץ אותה השלבים הם:

1. התקן את הספריות הנדרשות מהקובץ requirements.txt בתקייה Chatbot_App
2. נווט לתקייה \home_simple_version\rag_llm
3. וודא ש Ollama רץ בכתובת הרצויה
4. בחר את המודל הרצוי והדמות הרצויה בקובץ המיין והרץ אותו

6.5.7 הרצת מערכת הצ'אט על הרובוט במעבדה

השלבים להרצת המערכת על הרובוט דומים מאוד להרצת המערכת בבית

1. הדלק את ARI
2. התקן את הספריות הנדרשות מהקובץ requirements.txt בתקייה Chatbot_App
3. נווט לתקייה \Chatbot_App\ros_lab_version והרץ את הפקודה catkin_make ואת הפקודה source devel/setup.bash
4. שנה את ה Path למפתח ה ASR בתחילת הקובץ asr_transcriber.py שנמצא בתקייה audio_transcriber\scripts\
5. שנה את הרשאות קצבים בתקיית scripts שיהיה ניתן להריץ אותם, בעזרת הפקודה chmod +x src/audio_transcriber/scripts/*
6. וודא ש Ollama רץ בכתובת הרצויה, הכתובת הרצויה נמצאת ב llm_app.py וב rag_database.py בתקייה \src\audio_transcriber\rag_llm\ (במעבדה Ollama רץ על השואב אבק)
7. הרץ את אפליקציית ה web_app\app.py web מתוך התקייה הנוכחית (ros_home_version)

8. פתח את הכתובת <http://127.0.0.1:8000> ולחץ על כפתור ה Start Command להתחיל את התוכנית(בעתיד מסך זה יופיע על ARI)
9. כעת אפשר לדבר עם ARI, אם לא שומעים כלום יכול להיות שצריך להגביר את Volumen שלו.

6.6 תיאור הGIT של הפרויקט

קישור לגיט: https://github.com/mordeLash/chat_bot_project
 בתקיייה הראשית של גיט הפרויקט יש שני תקיות אחת לכל מערכת שבינינו

1. התקיה Benchmark_Tester

כדי לפשט אותו קצת נסמן שם כללי למבחן \$test \$model ופועל יש אחד כזה לכל מבחן ומודל וכן \$ref כאשר בפועל יש גם ref_inc. מבנה התקיייה נראה כך:

Benchmark Tester/

|— code/

- | |— main.py # הקובץ איתו מריצים את המבחנים
- | |— test_handler.py # מחלקה שמטפלת בתהליך הבחינה(מטעינה עד שמירת התשובות)
- | |— benchmark_handler.py # מחלקה שמטפלת בהרצת המבחן על מודל
- | |— dataset_handler.py # מחלקה שמטפלת בטעינת המבחנים
- | |— LLM_handler.py # קובץ לפישוט האינטרקציה עם מודל שפה
- | |— results_saver. # קובץ לשמירת תוצאות המבחן

|— config/

- | |— benchmarks.yaml # קובץ הגדרות המבחנים

|— prompts/ # מכיל תיקיות שמכילות פרומפטים מתאימים לבנצ'מארקים השונים

|— \$test/

- | |— \$test_eng.txt # הפרומט המהונדס
- | |— \$test_Prompt_template.txt # הטמפלייט לשאלות המבחן
- | |— \$test_simp.txt # הפרומט הפשוט

|— REFERENCE/

- | |— \$ref_system.txt # פרומפט השופט
- | |— \$test_template.txt # הטמפלייט לשאלות השופט

|— results/ # תוצאות הבדיקות

| |— \$model

- | | |— \$test_responses.xlsx # תשובות המודל למבחן
- | | |— \$test_summary.xlsx # סיכום התוצאות למבחן

|— requirements.txt # הספריות הדרושות למערכת

1. התקיה Chatbot_App

מבנה התקיה נראה כך:

Chatbot_App/

- |— **ros_home_version/** # גרסא שרצה בבית שדורשת רוס
- |— **ros_lab_version/** # הגרסא שרצה במעבדה
- |— **simple_home_version/** # גרסא שרצה בבית שלא דורשת רוס
- |— **Video Examples/** # סרטונים של דוגמאות לשיחה עם ארי
- |— **requirements.txt** # הספריות הדרושות למערכת

המבנה של התקיות של גרסאות המערכת מאוד דומה ולכן נסביר רק את ההכי מורכת התקיה `ros_lab_version` המבנה שלה הוא כך:

Chatbot_App/

- |— **ros4hri_ws/**
 - | |— **devel/** # תקייה שנוצרת אוטומטית על ידי רוס
 - | |— **src/**
 - | | |— **audio_transcriber** # התקיה של הפקג' שלנו
 - | | |— **pal_msgs** # TTS שנודרש עבור
 - | | |— **respeaker_ros** # המידע מהמיקרופון
 - | |— **CMakeLists** # קובץ אוטומטי של ROS לבניית הפרוייקט

נמשיך בתקיית הפקג' `audio_transcriber` המבנה הוא כך:

audio_transcriber/

- |— **launch/**
 - | |— **audio_transcriber.launch** # קובץ הרצת הפקג'
- |— **rag_llm/** # מכילה את הקוד של מערכת ה RAG Chatbot
- |— **scripts/**
 - | |— **asr_transcriber.py** # צומת רוס שמטפלת ב ASR
 - | |— **llm_node.py** # צומת רוס שמטפלת בתקשורת עם מערכת ה RAG Chatbot

מבנה התקייה rag_llm נראה כך:

rag_llm/

- |— **character_prompts/** # מכיל את הפרומטים של הדמויות השונות
- |— **faiss_index/** # מכיל את מאגר המידע הווקטורי
- |— **rag_data/** # הקבצים מהם בנינו את מאגר המידע
- |— chat_log.txt # מעקב אחר השיחות
- |— history_manager.py # מחלקה לניהול היסטוריית השיחה
- |— llm_app.py # מחלקה לניהול השיחה
- |— main.py # קובץ להפעלת המערכת ללא ROS
- |— rag_database.py # מחלקה לבניית והפעלת הRAG
- |— rag_maker.py # קובץ להרצת בניית מאגר מידע ווקטורי

7 ביבליוגרפיה

1. <https://www.elastic.co/what-is/large-language-models>
2. <https://www.geeksforgeeks.org/artificial-intelligence/what-are-llm-parameters>
3. https://www.vellum.ai/blog/how-should-i-manage-memory-for-my-llm-chatbot?utm_source=chatgpt&utm_medium=geo
4. <https://symflower.com/en/company/blog/2024/llm-benchmarks/#gpqa>
5. <https://www.kaggle.com/whitepaper-prompt-engineering>
6. <https://nexla.com/ai-infrastructure/prompt-engineering-vs-fine-tuning>
7. <https://www.ibm.com/think/topics/prompt-engineering-techniques>
8. <https://www.ibm.com/think/topics/retrieval-augmented-generation>

תמונות

1. <https://pal-robotics.com/robot/ari>
2. <https://medium.com/data-science-at-microsoft/how-large-language-models-work-91c362f5b78f>
3. <https://www.geeksforgeeks.org/artificial-intelligence/what-are-llm-parameters>
4. <https://inside-machinelearning.com/en/special-tokens>
5. <https://ollama.com/library/deepseek-r1>
6. https://cookbook.openai.com/examples/evaluation/evaluate_rag_with_llamaindex